

Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

First Author^{1*} and Second Author¹

^{1*}Department, Organization, City, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;

Contributing authors: second.author@example.com;

Abstract

Sequential approximate Bayesian computation (ABC-SMC/PMC) is the standard tool for likelihood-free inference, but its generation-staged structure imposes synchronization barriers that waste compute when simulator runtimes vary across parameters, precisely the regime of large heterogeneous high-performance computing (HPC) workloads. We introduce a *generation-free, single-arrival-driven* ABC algorithm that combines smooth-kernel ABC with a streaming limit of Adaptive Multiple Importance Sampling (AMIS): the proposal mixture and importance weights update after *every* evaluated particle rather than at population boundaries. The estimator it reports is a pure function of the evaluated history, so a crashed run is recovered by replaying its log rather than by checkpointing sampler state, and the propagator carries no archive between calls — a natural fit for the barrier-free island model of the Propulate optimization engine. We prove consistency for the estimator the experiments actually report, and a central limit theorem — with explicit rate conditions, an explicit asymptotic variance, and hence per-run confidence intervals — for damped-adaptation variants of it, the reported configuration being one we measure and find does not meet the central limit theorem’s rate condition. Both rest on a single condition on the algorithm: that the denominator it divides by asymptotically match the density its sample was drawn from, in a ratio $\mathbf{r}$ the results carry rather than assume away. A prior floor in that denominator bounds every importance weight deterministically, which removes the density-ratio assumption a local archive proposal cannot satisfy; $\mathbf{r} \equiv \mathbf{1}$ recovers the smooth-ABC posterior exactly. We decompose $\mathbf{r}$ into four factors and estimate two of them from stored runs — the reported posterior lies within roughly one percent in total variation of a reference that removes those two — while stating plainly that the results are conditional: the assumptions are

not established for the runs we report. Calibration of the reported posterior is verified directly by simulation-based calibration. Empirically, against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule (and, on all three benchmarks where we report systems gains, against a barrierized twin of our own propagator that isolates synchronization as the only difference in execution, though not at a matched evaluation budget), the method (i) is well calibrated in low dimensions and robust to multimodality — simulation-based calibration of the reported full-history estimator yields near-nominal coverage and the top- k archive retains both modes of a bimodal posterior in $\approx 86\%$ of trials — and conservative rather than over-confident in higher dimensions; and (ii) delivers substantial HPC benefits: under a persistent straggler the synchronous baseline’s throughput collapses by more than two orders of magnitude while the asynchronous method holds essentially flat, and under runtime heterogeneity it sustains several-fold higher simulation throughput at a fixed wall-clock budget. Scoring each method on its *own* reported posterior against a reference posterior — rather than on the distance from a truncated archive to a point mass at the true parameter, a statistic that rewards a collapsed posterior and is floored at the posterior’s own spread — leaves the asynchronous method and the matched baseline indistinguishable where a reference exists, and exposes what bounds both: the reported posterior’s effective sample size is set by the archive size rather than the simulation budget (**237–303** effective particles out of **1.1–1.3** million evaluated, flat across a thirteen-fold growth of the history), so barrier removal buys evaluations reliably and posterior accuracy only up to that ceiling. On a realistic Cellular Potts tissue-simulation workload the asynchronous method scales almost linearly to eight nodes, sustaining several-fold higher throughput ($\approx 5\times$ at **384** workers, widening with scale) than a synchronous baseline that, given the same core budget and a matching population, scales only sub-linearly, at a small cost in posterior quality on that benchmark — a benchmark assembled to exercise heterogeneous runtimes, whose summaries, discrepancy and parameter ranges were not tuned for identifiability and whose parameters confound, so its quality figures bound no method’s attainable accuracy and are not read here as an inference-quality verdict.

Keywords: approximate Bayesian computation, sequential Monte Carlo, adaptive multiple importance sampling, asynchronous algorithms, high-performance computing, likelihood-free inference

1 Introduction

Approximate Bayesian computation (ABC) is a standard tool for simulator-based inference when the likelihood is unavailable or too expensive to evaluate directly [1]. Sequential ABC methods improve efficiency by evolving proposal distributions and tolerance thresholds across a sequence of intermediate targets [2–5]. In practice, however, most ABC-SMC/PMC implementations remain organized around *synchronous* populations: the proposal and tolerance update only after an entire generation has been evaluated. Because simulator runtimes often depend strongly on the sampled

parameters, generation barriers leave compute nodes idle waiting for the slowest simulation, a dominant inefficiency on large heterogeneous high-performance computing (HPC) systems.

This paper introduces a generation-free alternative designed for exactly that regime and implemented within the barrier-free island model of the Propulate engine [6]. Our contributions are:

1. A **generation-free, single-arrival-driven ABC algorithm** that combines smooth-kernel ABC [7, 8] with a streaming limit of Adaptive Multiple Importance Sampling (AMIS) [9]. To our knowledge it is the first ABC algorithm whose proposal mixture updates after every evaluated particle rather than at generation/stage boundaries.
2. A **history-reconstructed particle archive** that makes the *reported estimator* a pure function of the evaluated history, and therefore recoverable by replay after a crash rather than by checkpointing.
3. **Integration into Propulate**, exploiting its barrier-free island model for true asynchronous execution on HPC.
4. **Asymptotic guarantees for the estimator we report.** We prove consistency and a central limit theorem for the reported estimator itself (§4, proofs in Appendix A): self-contained rather than a corollary of AMIS theory, with explicit rate conditions, an explicit asymptotic variance that separates the simulator’s own noise from the importance-sampling variance, and a consistent variance estimator giving per-run confidence intervals at no extra simulation cost. The whole of the algorithm’s contribution is isolated in one quantity — the limiting ratio r between the density the sample was drawn from and the denominator the estimator divides by — which appears in the results rather than as a caveat beside them and is exactly 1 when the two agree. We decompose r into four factors and measure two of them on stored runs, and we measure the stabilization rate the central limit theorem needs, finding the reported configuration misses it — the proposal’s distance to its limit decaying as $\tau^{-0.14}$ to $\tau^{-0.33}$ against the $\tau^{-1/2}$ required, while its per-step drift decays at the $O(k \log n)$ record rate as predicted. A prior floor in the denominator bounds every importance weight by $1/\delta$ whatever the sampler does, replacing the two-sided density-ratio condition a local archive proposal cannot meet.
5. An **apples-to-apples empirical evaluation against pyABC** [10, 11] in which pyABC’s acceptor is replaced by a probabilistic-rejection acceptor using the *same* ABC kernel and bandwidth schedule. This matched-kernel baseline removes the acceptance rule as a confound, and a **barrierized twin** of our own propagator — identical in every respect but the collective barrier before each breed — isolates synchronization directly on every benchmark where we report a systems gain: 21–402× slower than the asynchronous arm under a persistent straggler (at either of two barrier granularities, which agree to three significant figures wherever the straggler binds, so the gap is not an artifact of how finely the twin synchronizes), 1.1–25× under runtime heterogeneity, and 1.9–2.8× on the Cellular Potts workload (§7.1, §7.3); we separately verify (via SBC and matched-budget posterior error) that the reported streaming (full-history AMIS) posterior is comparable to the

baseline’s in low dimensions and under multimodality, and conservative rather than over-confident in higher dimensions.

The remainder of the paper summarizes the relevant ABC and importance-sampling background (§2), develops the method (§3) and its theory (§4), describes the implementation (§5), lays out the experimental program (§6), and reports results (§7).

2 Background and Related Work

2.1 Approximate Bayesian computation

ABC replaces exact likelihood evaluation with simulation and discrepancy-based acceptance. For observed data y and parameter θ , the ABC target at bandwidth ϵ is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta) L_\epsilon(\theta), \quad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho|\theta)}[K_\epsilon(\rho)], \quad (1)$$

where ρ is the discrepancy between simulated and observed summaries, π is the prior, and K_ϵ is the ABC kernel. Classical ABC uses the hard indicator $K_\epsilon(\rho) = \mathbf{1}[\rho < \epsilon]$; *smooth-kernel* ABC [7, 8] replaces this with a continuous kernel (e.g. Gaussian $\exp(-\rho^2/2\epsilon^2)$ or Epanechnikov), which removes the discontinuity in the likelihood approximation and admits standard SMC-sampler theory.

2.2 Sequential ABC algorithms

The generation-staged family (ABC-SMC [2, 3], ABC-PMC [4], and adaptive variants [5, 12, 13]) advances through intermediate targets using mixture proposals

$$q_r(\theta) = \sum_{j=1}^{N_r} W_j^{(r)} K(\theta \mid \theta_j^{(r)}) \quad (2)$$

built from accepted particles of the previous stage. The defining constraint is the *generation barrier*: the importance weights at stage r are computed against a single proposal q_{r-1} , which requires the full stage- r population before q_r can be formed.

2.3 Adaptive multiple importance sampling

AMIS [9] generalizes staged importance sampling: at each stage the proposal is adapted from past particles, and crucially *all* past particles are re-weighted against the cumulative proposal mixture $\frac{1}{n} \sum_{j=1}^n q_{\tau_j}$ (the balance heuristic of [14]) rather than against the proposal under which each was originally drawn. AMIS is consistent and satisfies a CLT under regularity conditions. Our method takes the *streaming limit* of AMIS: stage size one, single-arrival-driven adaptation.

2.4 Asynchronous SMC and parallel ABC

Our proposal-adaptation machinery descends from the adaptive-importance-sampling line — population Monte Carlo [15], AMIS [9] and its recycling-consistency theory

[16], surveyed in [17] — which we take to its streaming, single-arrival limit. Parallel and off-barrier SMC has been studied for state-space models — the island particle model [18], asynchronous anytime particle methods [19], and parallel resampling [20] — but none targets ABC, and none combines barrier-free execution with AMIS-style cumulative-mixture reweighting. Distributed ABC frameworks such as pyABC [10, 11], ABCpy [21], and ELFI [22] parallelize *simulation* effectively within a generation.

The closest work to ours is look-ahead scheduling [23], and we want to be precise about what it already does, because it does more than the framing above would suggest. It attacks exactly the idle time we do: rather than let workers stall while generation $t-1$ drains, it starts generation- t simulations preemptively on the free workers, from a proposal built either from the preliminary population of the first N acceptances (LA PREL) or from the last finalised population $\mathcal{P}_{t-2}$ (LA PAST), and importance-weights each preliminary draw by $\pi(\theta)/g_t(\theta)$ against whichever proposal produced it. It also identifies, and corrects for, the runtime-dependent selection we take up in §4.1: sampling from the preliminary population biases towards parameters that simulate quickly, which the authors demonstrate on a bimodal model with an artificially slow mode and remove either by proposing from $\mathcal{P}_{t-2}$ instead, or by closing a generation on the N accepted particles that were *started* first rather than the ones that finished first. That start-order rule is the same object our proofs need and our implementation does not have (§A.5).

Two things distinguish our setting. The generation *structure* is retained there and removed here: look-ahead fills the drain window, but a population is still declared, a proposal is still fitted once per population, and the weight is still against a single per-generation proposal g_t ; we have no populations, adapt after every arrival, and weight against the cumulative mixture of all proposals used so far. And the speed-ups differ in kind accordingly — theirs are the 10–50% recoverable from idle time at the boundary, ours are the one-to-two orders of magnitude available when the boundary itself is what costs (§7.1). The two are complementary rather than competing, and we do not claim that established frameworks simply stall at the barrier: one of them already does not. The trade-off we take on is that standard generational SMC theory no longer applies at all, which motivates §4.

3 Method

3.1 Design constraints and history-based state

We design the algorithm so that the *reported estimator* is a pure function of the evaluated history: the archive, the reconstructed bandwidth and the posterior weights are all recomputed from the recorded log, so a crashed run is recovered by replaying that log rather than by checkpointing sampler state, and the estimator is identical whether it is formed online or afterwards.

That is a property of the estimator, not of the sampler, and the distinction matters. The running propagator does carry state between calls, and it is not merely book-keeping: the online AMIS snapshot buffer enters the importance denominator of every candidate it proposes, that denominator becomes the candidate’s stored weight, and those weights are what the archive’s proposal mixture is built from — so the buffer

reaches the proposals, and a restart, which rebuilds it empty, therefore changes the sampler’s subsequent trajectory. Two scheduler throttles gate when the bandwidth is allowed to move and do the same. What survives a restart exactly is the estimator over whatever history was in fact evaluated; what does not is the identity of the trajectory (§C). We are explicit about this because “stateless” would otherwise overclaim: the estimator is reconstructible, the run is not replayable. For an efficient implementation we leverage the existing Propulate framework, whose propagator exposes a single call `__call__(inds) -> Individual`: it receives the entire evaluated history and emits the next candidate without persistent archive state or assimilation callbacks, exactly the history-based interface our design calls for. The archive, the bandwidth and the reported weights are therefore reconstructed from history rather than carried. Define the evaluated history at call n as

$$\mathcal{H}_n = \{(\theta_i, \rho_i, \epsilon_i, \tau_i, w_i)\}_{i=1}^n, \quad (3)$$

where θ_i is the parameter, ρ_i the discrepancy, ϵ_i the bandwidth active when θ_i was proposed, τ_i the proposal-time index, and w_i the stored core importance weight π/q_{τ_i} .

3.2 Tolerance reconstruction and monotonicity

Each proposed individual stores the bandwidth active at proposal time. The effective bandwidth reconstructed from history is the running minimum of these stored bandwidths, $\epsilon_{\text{hist}} = \min_i \epsilon_i$ (falling back to the initial tolerance on an empty history). The scheduler proposes ϵ_{sched} , and the bandwidth used is $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$, giving a monotone-decrease guarantee with no mutable propagator state.

3.3 Archive and smooth-kernel proposal

The reconstructed archive is the top- k history members by lowest discrepancy, $A_n = \text{Top}_k(\{\theta_i\}, \text{order by } \rho_i)$. The proposal mixture uses kernel-weighted archive members,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_{\Sigma}(\theta - \theta_j), \quad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \quad (4)$$

with $\sum_j \tilde{W}_j^{(n)} = 1$. The perturbation kernel K_{Σ} is Gaussian with covariance $\Sigma_n = s \widehat{\text{Cov}}_{\tilde{W}}(A_n)$ estimated from the kernel-weighted archive (s is the `perturbation_scale` hyperparameter), factored once via Cholesky. Smooth kernels remove the prior-vs-archive discontinuity present in hard-threshold ABC-PMC: every evaluated particle contributes continuously through its kernel weight.

3.4 Streaming AMIS importance weight

A particle θ^* drawn from q_n receives an importance weight under the balance heuristic over a sliding ring buffer $\mathcal{S}$ of past proposals (size S , sampled every `amis_interval`

calls):

$$w^* = \frac{\pi(\theta^*)}{\bar{q}_n^{\text{on}}(\theta^*)}, \quad \bar{q}_n^{\text{on}}(\theta) = \frac{1}{1 + |\mathcal{S}|} \left[q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta) \right]. \quad (5)$$

Setting $|\mathcal{S}| = 0$ recovers single-current-proposal weighting; increasing S enriches the balance-heuristic denominator toward the full cumulative mixture, which lowers the variance of the importance weights at a modest additional per-call cost. We use $S = 20$ by default, a value that captures most of that variance reduction while keeping each update cheap. A particle’s weight, assigned at proposal time, is corrected by the cumulative-mixture denominator at every subsequent use (the retroactive AMIS step). Because each snapshot q_s is the proposal reconstructed from the history truncated at call s , the buffer $\mathcal{S}$ is itself a deterministic function of $\mathcal{H}_n$, so the propagator stores nothing between calls. Equation (5) is the proposal-time importance weight; the *posterior* estimator reported in §4 reweights it by the smooth ABC likelihood $K_{\epsilon_n}(\rho)$ (8).

Three weight objects.

To avoid conflation, the method uses three distinct weightings. (i) The *adaptation weights* $\tilde{W}_j^{(n)}$ (4) are kernel weights over the top- k archive; they define the next proposal mixture q_n and so drive proposal *adaptation* only. (ii) The *proposal-time importance weight* w^* (5) is the balance-heuristic weight over the bounded sliding buffer $\mathcal{S}$ ($S = 20$), assigned online; it enters proposal adaptation through (4) and the effective-sample-size diagnostic, but never the reported posterior. (iii) The *reported posterior* is the retroactive estimator $\hat{\pi}_n$ (8), in which every evaluated particle is reweighted by $\pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i)$, where the *retroactive* denominator $\bar{q}_n$ — a different object from the online $\bar{q}_n^{\text{on}}$ of (5), and the one §4 analyses — is a draw-proportional mixture of $m \leq S+1 = 21$ history-reconstructed proposals plus a defensive prior component (Appendix B) — a fixed-size stand-in for the mixture the sample was actually drawn from, not the mixture over every evaluated particle; §4 states precisely what that substitution costs. The reported posterior densities and the SBC calibration are computed from it; the Wasserstein-vs-time quality curves are computed on the top- k archive (asynchronous) and the final population (synchronous), matched in size.

Stored fields and weight lifecycle.

Each evaluated individual i records exactly four fields: its parameter θ_i , its discrepancy ρ_i , the bandwidth ϵ_{τ_i} active when it was proposed, and its proposal-time weight w_i^* (5); the history $\mathcal{H}_n$ is the arrival-ordered sequence of these records. From $\mathcal{H}_n$ alone: the archive is the top- k members by smallest ρ under the reconstructed running-minimum bandwidth; the adaptation weights $\tilde{W}_j^{(n)}$ (weight (i)) are *recomputed* each call from the current archive (never stored) and define q_n ; the proposal-time weight w_i^* (weight (ii)) is computed once at proposal time over the sliding buffer $\mathcal{S}$, stored, and consumed by the ESS diagnostic and by parent selection during adaptation; and the reported posterior weight (weight (iii)) is *recomputed* retroactively from $\mathcal{H}_n$ using the $m \leq S$ snapshot denominator with a defensive prior floor. Only w_i^* is persisted; weights (i) and (iii) are pure functions of $\mathcal{H}_n$ recomputed on demand, so after a crash

the archive and the reported estimator are reconstructed from the recorded history, while the online buffer and the two scheduler throttles rebuild empty and affect only the proposal-time diagnostic.

3.5 Update loop

Algorithm 1 assembles the pieces above into the single-arrival update the propagator runs on each call. Every quantity it uses (the effective bandwidth, the archive, the proposal mixture, and the AMIS denominator) is reconstructed from the history $\mathcal{H}_n$ passed in, so the update carries no state of its own between calls; until the history holds at least k members it emits uniform prior draws to seed the archive.

Algorithm 1 History-reconstructed, single-arrival-driven ABC update

Require: History $\mathcal{H}_n$, archive size k , initial bandwidth ϵ_0 , snapshot buffer $\mathcal{S}$

- 1: Reconstruct ϵ_{hist} from stored tolerances; compute ϵ_{sched} ; set $\epsilon_n \leftarrow \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$
- 2: **if** $|\mathcal{H}_n| < k$ **then**
- 3: emit a uniform prior draw (bootstrap) $\triangleright$ covers $\bar{q}$ on the whole box
- 4: **else**
- 5: select archive A_n (top- k by ρ); form $\tilde{W}^{(n)}$ in log-space via K_{ϵ_n}
- 6: build Σ_n , Cholesky-factor once; sample θ^* by perturbing a $\tilde{W}^{(n)}$ -weighted parent
- 7: compute w^* via the AMIS denominator (5); periodically snapshot q_n into $\mathcal{S}$
- 8: **end if**
- 9: **return** θ^* for external simulation and discrepancy evaluation

3.6 Relation to classical ABC-SMC

Table 1 sets the method against a typical generation-staged ABC-SMC/PMC implementation — concretely the pyABC baseline used in our experiments — property by property. Such an implementation advances a fixed population through discrete generations behind a synchronization barrier, weights each stage against the single previous proposal, and carries an explicit mutable archive; none of these is intrinsic to ABC-SMC in general (smooth kernels and richer proposals exist), but each characterizes the synchronous baseline we compare against. The proposed method replaces each of these with a barrier-free counterpart: a single-arrival update in place of the generation step, a sliding top- k archive reconstructed from history in place of the explicit population, a smooth ABC kernel in place of the hard threshold, and a balance-heuristic weight over the cumulative mixture in place of single-proposal weighting. The decisive difference is the last row: because all state is a pure function of the history, there is nothing to synchronize between workers, which is what makes the method a natural fit for the asynchronous island model of §5.

Table 1 A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize the configured baseline, not ABC-SMC as a family.

Property	Classical ABC-SMC	This work
Update style	generation, barrier-synchronized	event-driven (single-arrival), no barrier
Archive	explicit, generational	reconstructed, sliding top- k
ABC likelihood	hard threshold	smooth kernel
Importance weight	against q_{t-1} only	balance heuristic over $\mathcal{S}$
Archive state	mutable, carried	none; reconstructed per call
Reported estimator	final population	pure function of the history

4 Theoretical Analysis

We prove consistency and a central limit theorem for the estimator this paper reports, computed by the algorithm this paper runs. The results are *conditional*, and it is worth being blunt about what that means here: the assumptions below are stated precisely, decomposed, and in part measured, but none of them is *established* for the configuration the experiments use. What the analysis buys is not that the conditions are discharged; it is that the algorithm’s departures from the ideal are carried inside the theorem statements as a single quantity, where they can be decomposed and estimated, rather than set beside them as a list of caveats. The proofs are in Appendix A; they are self-contained rather than corollaries of published AMIS theory, whose consistency results for recycling schemes [16] restrict the adaptation so that the weighting stage decouples — which a single-arrival scheme does not do — and whose original convergence argument [9] is heuristic.

The analysis is organized around a single question, because that is all the proofs turn out to need: *is the denominator the estimator divides by the density its sample was actually drawn from?* Write $\tilde{q}_i$ for the conditional density of the i -th candidate given the past, in the sense of the filtration fixed by Assumption 4. It absorbs every internal departure of the implementation — the top- k archive rule, the capped in-box rejection, the underflow redraws, the bootstrap phase — and it is the *only* way the sampler enters Theorem 1. That such a conditional density exists with respect to a filtration the proofs can use is itself an assumption once execution is asynchronous, which is why Assumption 4 states it rather than leaving it implicit. Sequentially it is exactly the density the propagator emits from; asynchronously the two can differ, and Assumption 4 is where that is confronted rather than hidden. Write

$$\bar{q}_n^\star := \frac{1}{n} \sum_{i \leq n} \tilde{q}_i \quad (6)$$

for the mixture the pooled sample was drawn from, against the reported denominator

$$\bar{q}_n = w_n \pi + (1 - w_n) \sum_{s=1}^m \alpha_{s,n} q_{\tau_s}, \quad w_n = \max(\nu_n, \delta), \quad \alpha_{s,n} \geq 0, \quad \sum_s \alpha_{s,n} = 1, \quad (7)$$

the draw-proportional deterministic mixture [14, 24] of $m \leq S+1 = 21$ history-reconstructed proposals with a prior component whose mass is the observed bootstrap share ν_n floored at $\delta = 0.5/(m+1)$, exactly as the implementation builds it (Appendix B). Throughout, ϵ_∞ denotes the limit of the bandwidth schedule $\{\epsilon_n\}$, whose existence and positivity are part of Assumption 5(i); $L_\epsilon(\theta) = \mathbb{E}[K_\epsilon(\rho) \mid \theta]$ is the smooth ABC likelihood (1), $L_\epsilon^{(2)}(\theta) = \mathbb{E}[K_\epsilon(\rho)^2 \mid \theta]$, and the estimator is

$$\hat{\pi}_n(f) = \frac{\sum_{i=1}^n W_{i,n} f(\theta_i)}{\sum_{i=1}^n W_{i,n}}, \quad W_{i,n} = \frac{\pi(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}. \quad (8)$$

What is and is not new here.

The r -tilted limit of Theorem 1 is, in itself, the elementary observation that self-normalized importance sampling against a mismatched denominator converges to a correspondingly tilted target; stated for a fixed proposal it would not merit a theorem. Three things are not routine. The scheme is adaptive at every draw, so the argument runs through a martingale array rather than an i.i.d. one, and the available AMIS results do not cover it: [9]’s convergence argument is heuristic, and [16] restricts the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do. The prior floor in the *denominator* bounds every weight by $1/\delta$ with no moment or density-ratio condition, which is what makes the conditional Lindeberg step trivial and lets a local archive proposal — for which the usual two-sided ratio bound simply fails — be used at all. And the departure from the ideal is carried as r inside the statements, then decomposed and partly measured, rather than assumed away. We do not claim to weaken the hypotheses of any existing AMIS theorem; we claim a different bargain, in which the implementation’s infidelity is a quantity in the conclusion instead of a condition on the algorithm.

Assumptions.

Assumption 1 (kernel). $K_\epsilon : [0, \infty) \rightarrow [0, 1]$ is measurable with $K_\epsilon(0) = 1$ (unit *peak*, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and for every $a \in (0, \epsilon_0]$ there is $L_K(a) < \infty$ with $|K_\epsilon(\rho) - K_{\epsilon'}(\rho)| \leq L_K(a)|\epsilon - \epsilon'|$ for all $\rho \geq 0$ and $\epsilon, \epsilon' \in [a, \epsilon_0]$. The limiting target is non-degenerate: $Z_{\epsilon_\infty} = \int_\Theta \pi L_{\epsilon_\infty} > 0$.

Assumption 2 (denominator class). $\Theta \subset \mathbb{R}^d$ is a non-degenerate axis-aligned box and the prior density satisfies $0 < \pi_- \leq \pi \leq \pi_+ < \infty$ on it. Each q_{τ_s} in (7) is a positive function on Θ of truncated-Gaussian-mixture form — at most k components, means in Θ , weights in the simplex, covariance Σ with $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ for fixed $0 < \sigma_- \leq \sigma_+ < \infty$, and per-component normalizing constants evaluated as the implementation evaluates them. Nothing requires $\bar{q}_n$ to integrate to one: the proofs use only positivity, boundedness and Lipschitz dependence on the parameters, so an approximate normalizing constant leaves Assumption 2 intact and shows up instead as a contribution to r .

Assumption 3 (prior floor in the denominator). $\delta \in (0, 1/2]$ is fixed and $w_n \geq \delta$ in (7).

Assumption 4 (adapted filtration). There is a filtration $(\mathcal{F}_i)$ with $\xi_i = (\theta_i, \rho_i)$ $\mathcal{F}_i$ -measurable, $\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i$ and $\rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i)$. Under sequential execution this holds by construction; under asynchronous execution it is a genuine assumption (Appendix A).

Assumption 5 (fidelity and stabilization). *(i, consistency)* $\epsilon_n \downarrow \epsilon_\infty > 0$ almost surely (ϵ_∞ may be random), and the denominator tracks the draw mixture up to a limiting ratio: for some r with $0 < r_- \leq r \leq r_+ < \infty$,

$$\|\bar{q}_n^*/\bar{q}_n - r\|_\infty \longrightarrow 0 \quad \text{a.s.} \quad (9)$$

(ii, central limit theorem) In addition ϵ_∞ and r are deterministic, a (possibly random) limit $\tilde{q}_\infty$ of $\tilde{q}_i$ and $\bar{q}_\infty$ of $\bar{q}_n$ exist with $r = \tilde{q}_\infty/\bar{q}_\infty$, and, almost surely,

$$\begin{aligned} \sqrt{n} (|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) &\rightarrow 0, \\ \frac{1}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty) &\rightarrow 0, \end{aligned} \quad (10)$$

where ϵ_{i-1} and $\bar{q}_{i-1}$ are the bandwidth and the denominator (7) formed from the first $i-1$ draws (both known before draw i ; convention $\bar{q}_0 := \pi$).

Assumption 1 holds for the Gaussian kernel with $L_K(a) = 2/(ea)$ and Epanechnikov with $2/a$; it excludes the hard indicator, which is not Lipschitz in ϵ — one concrete reason the method uses smooth kernels. The Lipschitz constant is stated per compact sub-interval because it degrades as $\epsilon \downarrow 0$; no *deterministic* bandwidth floor is needed, since the proof localizes over $[1/j, \epsilon_0]$ and intersects countably many almost-sure events (Appendix A). Assumptions 2 and 3 constrain only the *reported denominator*, which the implementation builds to that exact recipe. Assumption 5 carries everything else. For *consistency*, (9) is the whole of the sampler’s contribution: not that the sampler take any particular form, but that whatever it does be asymptotically reflected in the denominator, up to a ratio r the results carry rather than assume away. The central limit theorem asks more — the per-draw rate in (10) and, through the variance, the limit $\tilde{q}_\infty$ itself — so the “one channel” reading is exact for Theorem 1 and approximate for Theorem 3. Note the two clauses differ in strength: consistency permits r and ϵ_∞ to be random, while the central limit theorem needs them deterministic, because the centred integrand must be a fixed function for the martingale arrays to be adapted. That restriction costs nothing in the case of most interest, $r \equiv 1$, which is deterministic no matter how run-dependent $\tilde{q}_\infty$ is.

Theorem 1 (consistency). *Let Assumptions 1–4 and 5(i) hold. Then for every bounded measurable $f : \Theta \rightarrow \mathbb{R}$,*

$$\hat{\pi}_n(f) \longrightarrow \pi_{\epsilon_\infty}^r(f) := \frac{\int_\Theta f \pi L_{\epsilon_\infty} r}{\int_\Theta \pi L_{\epsilon_\infty} r} \quad \text{almost surely.} \quad (11)$$

Corollary 2 (exact fidelity). *If $r \equiv 1$ — the denominator is asymptotically the density the sample was drawn from — then $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}(f)$, the smooth-ABC posterior at the limiting bandwidth.*

Theorem 3 (central limit theorem). *Let Assumptions 1–4 and 5(i)–(ii) hold. Then for every bounded measurable f ,*

$$\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) \rightarrow \sigma_f \mathcal{Z} \quad \text{stably}, \quad \mathcal{Z} \sim \mathcal{N}(0, 1) \text{ independent of } \mathcal{F}_\infty, \quad (12)$$

where the $\mathcal{F}_\infty$ -measurable σ_f^2 is

$$\sigma_f^2 = \frac{1}{Z_r^2} \int_{\Theta} \pi(\theta)^2 (f(\theta) - \pi_{\epsilon_\infty}^r(f))^2 L_{\epsilon_\infty}^{(2)}(\theta) \frac{\tilde{q}_\infty(\theta)}{\bar{q}_\infty(\theta)^2} d\theta, \quad Z_r = \int_{\Theta} \pi L_{\epsilon_\infty} r. \quad (13)$$

A caveat belongs with this statement rather than after it: the rate conditions (10) are measurably violated in the configuration our experiments use (§4.1), so Theorem 3 should be read as covering the damped- or frozen-adaptation variants, not those runs. At $r \equiv 1$ the variance collapses to $Z_{\epsilon_\infty}^{-2} \int \pi^2(f - \pi_{\epsilon_\infty} f)^2 L_{\epsilon_\infty}^{(2)} / \bar{q}_\infty$, the familiar self-normalized form. Its structure is readable in either case. Because $L_\epsilon^{(2)} \geq L_\epsilon^2$ by Jensen, (13) always exceeds the variance of the corresponding *noise-free* importance sampler that could evaluate L_{ϵ_∞} exactly; the gap $L_\epsilon^{(2)} - L_\epsilon^2 = \text{Var}(K_\epsilon(\rho) \mid \theta)$ is precisely the price of running one simulation per proposed parameter. For the Gaussian kernel $K_\epsilon(\rho)^2 = K_{\epsilon/\sqrt{2}}(\rho)$, so $L_\epsilon^{(2)} = L_{\epsilon/\sqrt{2}}$ and that price is computable from the same likelihood at a tightened bandwidth. Convergence is stable rather than merely distributional because $\bar{q}_\infty$, and hence σ_f^2 , may be random; it reduces to an ordinary $\mathcal{N}(0, \sigma_f^2)$ limit when the limiting proposal is deterministic. Stability is also what makes the theorem usable, because it survives division by a consistent variance estimate:

Corollary 4 (per-run confidence intervals). *Under the assumptions of Theorem 3, and provided $\sigma_f^2 > 0$, with $\hat{\sigma}_n^2 := n \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2 / (\sum_i W_{i,n})^2$ we have $\hat{\sigma}_n^2 \rightarrow \sigma_f^2$ a.s. and $\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) / \hat{\sigma}_n \Rightarrow \mathcal{N}(0, 1)$. The interval $\hat{\pi}_n(f) \pm z_{1-\alpha/2} \hat{\sigma}_n / \sqrt{n}$ therefore has asymptotic level $1 - \alpha$ and is computable from the stored weights alone, at no extra simulation cost.*

Proposition 5 (how far the tilt can move the target). *Let $A \subseteq \Theta$ and suppose $|r - 1| \leq \eta_A$ on A and $|r - 1| \leq 1$ off it. Then with $\zeta := \eta_A + \pi_{\epsilon_\infty}(A^c)$,*

$$\zeta < 1 \implies \|\pi_{\epsilon_\infty}^r - \pi_{\epsilon_\infty}\|_{\text{TV}} \leq \frac{\zeta}{1 - \zeta}. \quad (14)$$

More sharply, ζ may be replaced by $\zeta_0 = \pi_{\epsilon_\infty}(|r - 1|)$. Estimating ζ_0 from a stored run requires untilting: the normalized reported weights $\hat{w}_i$ target π^r , not π_{ϵ_∞} , so $\sum_i \hat{w}_i |r(\theta_i) - 1|$ estimates $\pi^r(|r - 1|)$; weights proportional to $W_{i,n}/r(\theta_i)$ target π_{ϵ_∞} and give $\hat{\zeta}_0$.

4.1 What r is for this implementation

Everything specific to the code enters through r . To attribute it, walk from the density actually sampled from down to the function actually divided by, changing one thing at a time. Let $\bar{q}^{(1)} = \frac{1}{n} \sum_{i \leq n} q_i$ use the *nominal* archive proposals — the truncated mixtures the propagator intends, with exact normalizing constants and without the rejection cap or the underflow redraws; let $\bar{q}^{(2)}$ replace those constants by the implementation’s product-of-marginals values; and let $\bar{q}^{(3)}$ be the m -point draw-proportional version of $\bar{q}^{(2)}$ carrying the observed bootstrap share ν_n . Assume each of the four displayed ratios has a uniform limit, finite and bounded away from zero — which fidelity alone does not give, since (9) constrains only their product. Writing $\bar{q}$ for the reported denominator (7),

$$r = \underbrace{\lim \frac{\bar{q}^*}{\bar{q}^{(1)}}}_{\text{(d) cap, redraws, asynchrony}} \cdot \underbrace{\lim \frac{\bar{q}^{(1)}}{\bar{q}^{(2)}}}_{\text{(c) normalizer}} \cdot \underbrace{\lim \frac{\bar{q}^{(2)}}{\bar{q}^{(3)}}}_{\text{(b) quadrature}} \cdot \underbrace{\lim \frac{\bar{q}^{(3)}}{\bar{q}}}_{\text{(a) floor}}.$$

The product telescopes for any positive intermediates, so the content is in the choice of intermediates, not the algebra: a different insertion order attributes the total differently, and the factors are a decomposition of one quantity rather than four independent errors. Note also what the first factor carries under asynchronous execution. If completion times steer scheduling, $\tilde{q}_i$ is the enlarged-field conditional law rather than the density the propagator emitted (Assumption 4), and that discrepancy sits in $\bar{q}^*/\bar{q}^{(1)}$ alongside the cap and the redraws. The four factors are then not a complete attribution of r to implementation details — one of them is partly a property of the schedule.

(a) *The prior floor.* When genuine bootstrap draws carry less than a δ share of the history, $w_n = \delta > \nu_n$ and the denominator holds more prior mass than the sample does. With $u = q_\infty/\pi$ the limiting proposal-to-prior ratio, this contributes $r_{\text{floor}} = (\nu_\infty + (1 - \nu_\infty)u)/(\delta + (1 - \delta)u)$, so $r_{\text{floor}} - 1 = (\delta - \nu_\infty)(u - 1)/(\delta + (1 - \delta)u)$: it is exactly 1 when $\nu_\infty \geq \delta$, below 1 wherever the proposal is thinner than the prior, and $r_{\text{floor}} \rightarrow 1$ as $\delta \rightarrow 0$, i.e. as m grows. This is the one contribution available in closed form.

(b) *The snapshot quadrature.* The m -point mixture stands in for the n -point one, with error at most V_n/m where $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ is the total variation of the *reconstructed* proposal path; this controls the third factor above, not the fidelity of the whole. Fixing m therefore trades cost against fidelity; see the remark below.

(c) *The truncation constant.* Each mixture component is normalized by the product of its per-axis in-box masses, which is exact for a diagonal perturbation covariance and increasingly accurate as a component’s mass sits away from the faces of the box relative to its own scale, but an approximation otherwise.

(d) *Capped rejection, underflow redraws, and (asynchronously) the filtration gap.* A parent’s perturbation is rejection-sampled into the box for at most 992 attempts, so component j is realized with probability $W_j[1 - (1 - Z_j)^{992}]$ and the residual is emitted as a prior draw; and a candidate whose proposal-time denominator falls below 10^{-12} is redrawn up to five times, the last one being emitted regardless. The cap changes the law relative to unlimited rejection whenever $Z_j < 1$, through the residual probability

$(1 - Z_j)^{992}$; 10^{-3} is the scale at which that becomes appreciable rather than a threshold below which it is absent. Under asynchronous execution this factor additionally carries the gap between the emitted proposal density and the conditional law of Assumption 4, which is a property of the schedule; unlike the other three contributions it is neither measured nor bounded structurally, and enters only through the assumption that this factor has a finite, positive limit.

Measured, in part.

What follows measures two of the four factors, not r . Two of the four — the prior floor and the snapshot quadrature — can be estimated rather than bounded, because $\hat{\pi}_n$ is a post-hoc pass: recomputing (7) at a much larger m and at the observed prior share gives a reference against which those factors are evaluated at every particle. The remaining two are *not* captured by such a reference: the truncation constant is bounded rather than measured, and factor (d) is bounded only for its cap-and-redraw part, its asynchronous component resting on the assumed factor limit alone. The figures below are therefore a partial accounting, not a lower bound on the total: the factors are multiplicative and an unmeasured one could in principle offset a measured one. On the Cellular Potts production history (33,748 asynchronous evaluations across 48 workers, two correlated dimensions, so that (c) is live), contribution (b) alone gives $\hat{\zeta} = 0.022$ with $r \in [0.92, 1.80]$, and the reported posterior sits 1.1% in total variation from the large- m reference, with posterior means within 0.2% and 1.1% of a standard deviation and the effective sample size essentially unchanged (0.213 against 0.228). On a one-dimensional Gaussian-mean history where the exact n -point mixture is computable, (a) and (b) together give $\hat{\zeta} = 0.042$ and a total-variation distance of 0.26% from the reference that removes (a) and (b), with the reported posterior 0.5% narrower — the tail-suppressing direction predicted by (a). Of contribution (d), the two events a stored history records — a rejection fallback and an exhausted redraw — did not occur once in 12,000 draws; a redraw that succeeded on a later attempt leaves no trace. We do not rule those out: we observe only that triggering one requires a candidate to land where the mixture it was drawn from has density below 10^{-12} , which is a necessary condition for the event, not a bound on its probability. The bound (14) is honest but loose against these direct measurements, by about a factor of two on the Cellular Potts run (2.3% against 1.1%) and by an order of magnitude on the one-dimensional history (4.4% against 0.26%), as sup-norm-free bounds valid for every f generally are.

Do the reported runs meet the stabilization conditions?

Assumption 5(ii) is the one place where we can do better than assume, because the proposal sequence is reconstructible from a stored history. Rebuilding q_τ along the Cellular Potts production run and comparing against the terminal proposal separates two rates that behave very differently. The *per-step drift* $\|q_{\tau+1} - q_\tau\|_\infty$ decays as $\tau^{-0.92}$, and the archive’s membership changes only 832 times in 33,748 draws, tracking $k \log n$ at a ratio between 0.64 and 0.80 across the whole run — the rate a record-statistics heuristic predicts, since the k -th best of n draws improves with probability $\approx k/n$. The proposal path’s total variation is therefore $O(\log n)$, and the snapshot clause is comfortably met. The *distance to the limit* is another matter. Fitting $\|q_\tau -$

$q_\infty \|_\infty \sim \tau^{-b}$ gives $b \approx 0.33$ over the full range and $b \approx 0.14\text{--}0.23$ when the fit is restricted away from the end, where using the terminal proposal as a stand-in for q_∞ biases b upward. Assumption 5(ii) needs $b > \frac{1}{2}$; at the measured values the aggregate $n^{-1/2} \sum_{i \leq n} \|q_i - q_\infty\|_\infty$ grows like $n^{0.17}$ to $n^{0.36}$ instead of vanishing.

We therefore do not claim Theorem 3 for the configuration these experiments run: its rate condition is not merely unverified there but measurably violated. Theorem 1 is unaffected, needing no rate. Two things temper the negative result and point somewhere specific. The measurement is one replicate of one benchmark over about a decade and a half of n , against a stand-in limit, so it is a diagnostic and not an asymptotic statement. The natural explanation — that the run leaves `min_tol` unset, so ϵ_n was still tightening and the archive was chasing a moving target — we tested and can reject. Running a two-dimensional Gaussian-mean benchmark serially under the same propagator, once with the data-driven schedule and once with the bandwidth pinned at the tolerance the first run reached a third of the way in (a floor the schedule then sat on for 68% of archive-phase draws), moves b by less than 0.07 and in opposite directions depending on the fit range: $0.61 \rightarrow 0.67$ restricted, $0.76 \rightarrow 0.69$ over the full range. Both arms comfortably satisfy $b > \frac{1}{2}$. So a tightening bandwidth does not prevent stabilization, and cannot be what breaks the Cellular Potts run.

What does explain it is the target. The Cellular Potts configuration was assembled to exercise a realistic heterogeneous-runtime workload for the systems measurements of §7.3: its summaries, discrepancy and parameter ranges were not selected for identifiability, and its two parameters confound. Re-running the serial benchmark with that property deliberately induced — observing only $\theta_1 + \theta_2$, so the target is a ridge rather than a point — collapses the exponent from $b = 0.61\text{--}0.76$ to $b = 0.00\text{--}0.29$, squarely onto the Cellular Potts range. A confounded target reproduces the failure in a *serial* run, so asynchronous replay is not needed to account for it and the adaptation rule is not what breaks.

The mechanism is transparent once stated: on a ridge the top- k archive slides along the flat direction indefinitely, because there is always a marginally better point further along it, so the proposal never settles and $\|q_\tau - q_\infty\|_\infty$ does not decay at the required rate. Assumption 5(ii) is thus a condition on the *inference problem* as much as on the sampler: it holds comfortably for a well-identified target under the same top- k rule and the same bandwidth schedule, and fails when the summaries do not identify the parameters. We therefore state Theorem 3 for the damped- or frozen-adaptation variants when the target is weakly identified, and note that for the identified benchmarks the measured rate satisfies the condition with room to spare.

Fixed versus growing m .

Contribution (b) is where the estimator’s cost meets its fidelity. With m fixed, the m -point mixture tracks the n -point one only if the proposal path settles, the error being V_n/m . Letting $m = m_n \rightarrow \infty$ with $V_n/m_n \rightarrow 0$ removes *this* contribution to r — it does not by itself deliver $r \equiv 1$, since the other three remain — and the proofs go through for any $m_n = o(n/\log n)$ *provided the floor δ is held fixed rather than tied to m* . The implementation ties them, $\delta = 0.5/(m+1)$, so growing m there would send $\delta \rightarrow 0$, and with it the uniform weight bound $1/\delta$ of Lemma 6 that makes the

Lindeberg step trivial and fixes the constants in the law of large numbers; the rate conditions would have to be redone. Decoupling the two is a one-line change to the post-hoc pass and costs nothing (Appendix A, Remark 2), at a retroactive pass of $\mathcal{O}(nm_n k)$ instead of $\mathcal{O}(nk)$; the literal cumulative mixture is the endpoint $m_n = n$ at $\mathcal{O}(n^2 k)$. The measurements above locate the practical knee: on the Gaussian-mean history the reported posterior is within 3×10^{-4} in total variation of the exact n -point mixture by $m = 50$, so a $2.5\times$ more expensive post-hoc pass removes factor (b) almost entirely — which is not the same as achieving $r \equiv 1$, the other three factors being untouched. What the paper reports is the $m = 21$ end, and §5 gives its cost.

5 Implementation in Propulate

The algorithm is a Propulate propagator that carries no archive between calls: tolerance schedulers are pure functions of history; the archive is the top- k history members by smallest discrepancy under the reconstructed bandwidth; the smooth-kernel mixture weights are formed in log-space for numerical stability. The reported posterior is the retroactive AMIS estimator (8), computed off the timed inference path from the saved history (a pure function of it, hence identical whether computed online or after a crash/restart); the bare proposal-time weight (5) is retained separately for the effective-sample-size diagnostic. This retroactive step is a one-time pass of cost $\mathcal{O}(nk)$ in the n evaluated particles and archive size k , run after the timed budget and therefore excluded from the throughput measurements by design (the systems comparison is over simulation throughput). For cost-bearing simulators it is negligible against the simulation time; for near-instantaneous simulators whose history reaches $\mathcal{O}(10^7)$ particles it is non-trivial but bounded ($\mathcal{O}(nk)$ with the fixed $m \leq S+1 = 21$ snapshot count), and is evaluated in fixed-memory chunks (Appendix B) so it stays a bounded fraction of wall-clock and memory. For an apples-to-apples comparison, pyABC’s `UniformAcceptor` is replaced by a probabilistic-rejection acceptor using the same $K_\epsilon(\rho)$ and bandwidth schedule as the asynchronous side. The acceptor and kernel are thus matched across methods; the synchronous baseline differs in its generation-staged *execution* (the systems variable under test) and, on the inference side, in retaining classical population weighting rather than the streaming AMIS estimator, whose quality we check separately (§7.2). Distributed execution uses Propulate’s barrier-free island model over MPI; for wall-time-limited HPC experiments the synchronous baseline uses fixed populations/generations, which yields more interpretable comparisons than contrasting methods that stop under different ϵ rules.

6 Experimental Design

6.1 Research questions

(RQ1) Does the method recover posteriors comparable to established ABC baselines? **(RQ2)** Does asynchronous execution improve wall-clock efficiency and utilization under runtime heterogeneity? **(RQ3)** Does it scale on a realistic, expensive simulator, the regime where running a synchronous matched baseline would itself be prohibitively wasteful?

6.2 Benchmarks, baselines, and metrics

We use four benchmarks of increasing realism (Table 2): Gaussian-mean inference (analytic posterior; sanity and calibration), the g-and-k distribution (classical intractable-likelihood benchmark), the stochastic Lotka–Volterra system, and a Cellular Potts tissue model (`cellsInSilico`) as the realistic heterogeneous-runtime workload. Baselines are rejection ABC (small problems), the matched-kernel synchronous baseline (a probabilistic-rejection pyABC variant, the main comparator), and pyABC as an external reference. Statistical metrics: credible-interval coverage via simulation-based calibration (SBC); W_1 between each method’s *reported* posterior and a reference posterior, on the two benchmarks where a reference is available (Gaussian-mean analytically, g-and-k via the asymptotic law of its octile summaries); and, where no reference exists, W_1 between the top- k archive and a point mass at the true parameter. We keep the last of these clearly separated from the first two, because it does not measure what its name suggests: a point-mass target is minimised by a posterior that has *collapsed*, and it is floored at the posterior’s own spread, so it ranks an over-concentrated archive above a correct one and cannot resolve differences below that floor (§7.1). We read it as a concentration diagnostic and not as posterior quality.

One property of the Lotka–Volterra benchmark bounds what its statistical numbers can mean, and we state it here rather than leave it to be discovered. The simulator has an absorbing state: if either population reaches zero before $T_{\max}$ the run returns a fallback discrepancy, which a smooth kernel weights to effectively zero. At the configured parameters that is the usual outcome. Survival is 2.6% of 2000 simulations at the true parameter, and 1.7% over 400 uniform prior draws, of which 92.5% never survived one of 25 attempts. The observed dataset is itself survival-conditioned — it is re-drawn until a trajectory survives, which here takes more than eleven attempts — so the target is $p(\theta \mid s_{\text{obs}}, \text{survived})$ against a survival-conditioned s_{obs} , which is internally consistent: the extinct particles’ zero weight *is* the $P(\text{survive} \mid \theta)$ factor of the likelihood and belongs there. The consequences split. Throughput and scaling on this benchmark are unaffected, because an extinct simulation costs what a surviving one costs and that cost is the systems quantity under test. Posterior recovery is a different matter: the informative sample is $\approx 2\%$ of the nominal budget, so a 1.2×10^6 -evaluation run carries the information of $\approx 25,000$, and any method comparison on it is substantially a comparison of how each copes with a 98%-rejection regime. We therefore report Lotka–Volterra as a systems benchmark and treat its posterior curves as we treat Cellular Potts’s: as diagnostics that bound no method’s attainable accuracy. Computational metrics: simulation throughput, worker utilization / idle fraction, and wall-clock time to a fixed posterior-error target. Convergence curves are placed on a shared time grid via last-observation-carried-forward resampling so asynchronous (fine-grained) and synchronous (per-generation) records compare fairly.

6.3 HPC studies

Three dedicated studies probe the asynchrony claim: (i) *stochastic runtime heterogeneity* (a log-normal post-evaluation delay with sweep over spread σ); (ii) *straggler tolerance* (one worker given a permanent post-evaluation delay scaled by $0 \times$

Table 2 Benchmark suite and role in the evaluation.

Benchmark	Role	Outputs
Gaussian mean g-and-k	sanity / calibration intractable- likelihood	analytic-posterior error, SBC coverage posterior recovery, Wasserstein vs. budget
Lotka–Volterra	simulator-based ABC	posterior recovery, quality vs. time
Cellular Potts	realistic HPC work- load	throughput, utilization, scaling, posterior

(control), $1\times$, $5\times$, $10\times$, $20\times$); and (iii) *strong scaling* at fixed wall-clock budget over worker counts from 1 up to 288 (six fully-packed nodes) on Lotka–Volterra and up to 384 (eight nodes) on Cellular Potts, with up to five replicates each. A hyperparameter *sensitivity* grid (archive size, perturbation scale, tolerance scheduler, initial tolerance) and an *ablation* (hard vs. smooth kernel; with vs. without AMIS reweighting) complete the suite.

7 Results

Unless noted, results are five-replicate medians on JUWELS; figures compare the matched-kernel synchronous baseline (labelled *Synchronous*) against the proposed asynchronous method (*Asynchronous*, ours). We lead with the computational evidence (RQ2), the paper’s central claim, then verify that inference quality is not sacrificed (RQ1), and close with the realistic workload (RQ3).

7.1 Computational advantage (RQ2)

Straggler robustness.

This is the clearest illustration of the barrier’s cost (Fig. 1). One worker is given a permanent post-evaluation delay of 0.1 s scaled by $0\times$ (control), $1\times$, $5\times$, $10\times$, $20\times$. As the injected delay grows from the control to $20\times$, the synchronous baseline’s throughput collapses by more than two orders of magnitude (median $\approx 8800 \rightarrow 60$ simulations/s) because each generation blocks on the straggler, whereas the asynchronous method routes work to available workers and holds essentially flat ($\approx 3900 \rightarrow 3800$ simulation-s/s); only at the $0\times$ control is the baseline faster, where a generation barrier is cheap for this near-instant simulator. Posterior quality is not paid for this throughput: measured against the analytic posterior, the asynchronous method’s reported estimator sits at 0.008–0.011 at every slowdown and the baseline’s final-population estimator at 0.017–0.027 (§7.2). We read that gap as a difference in the estimators’ support — a weighted full history against a 100-particle final generation — rather than as a difference between the samplers.

Isolating synchronization: a barrierized twin.

The comparison above is against a different algorithm, so it cannot by itself attribute the gain to the barrier (§9, item vii). We therefore build a *barrierized twin* of our own method: the identical propagator — same proposals, same kernel weights, same archive rule, same reported estimator — with a collective barrier inserted immediately before each breed, so that the asynchronous arm and the twin differ in *nothing but* when a worker is permitted to proceed. Table 3 reports both on the straggler benchmark over five replicates. Only the twin needs a simulation-count budget: a collective barrier requires identical per-rank call counts, so a wall-clock stop — which halts ranks independently, as the first one to reach the deadline — deadlocks it. The asynchronous arm therefore runs wall-limited (300 s of active wall-clock) and the twin runs to a fixed simulation count, and we compare the two as *rates*: simulations per second of active wall-clock on both arms. The asynchronous arm holds $\approx 3100\text{--}3900$ simulations/s at every slowdown, reproducing the throughput of the main experiment above; the twin runs at 48.7 simulations/s already at the $0\times$ control — the cost of synchronizing a ≈ 4 ms simulator, where barrier latency alone dominates — and degrades to 8.0 as the straggler worsens, a factor of 402 below the asynchronous arm at $20\times$. Barrier removal is therefore worth between one and two-and-a-half orders of magnitude of throughput on this workload, measured against our own algorithm rather than inferred from a cross-algorithm comparison.

And posterior quality? It needs more care than the throughput does, because the statistic this comparison first used has no resolution on this benchmark. W_1 between the unweighted top- k archive and a point mass at the true parameter is indeed 0.069–0.085 on all three arms at every slowdown. But the analytic posterior here has standard deviation $\sigma_{\text{obs}}/\sqrt{n_{\text{obs}}} = 0.1$, so an *exactly correct* posterior scores 0.090 on that statistic: every arm is at or just below the score of a perfect answer, and the ones below it are there because a point-mass target rewards an over-concentrated archive. Agreement on that number is not evidence of anything, and we no longer report it. Evaluated instead on the estimator this paper actually reports — the full-history AMIS posterior, against the analytic posterior — the arms separate sharply (Table 3). Through $5\times$ all three agree at 0.008–0.015. At $10\times$ the fine twin’s reported posterior moves to 0.74 and at $20\times$ to 1.73, with the coarse twin following one step behind at 1.07; the asynchronous arm stays at 0.009–0.013 throughout. The twins’ *archives* stay on target in every one of these runs — it is the weights that degenerate, not the particles.

We do not read this as the barrier corrupting the estimator, because the two arms are not at a matched budget: at $20\times$ the twin completes 3,200 evaluations against the asynchronous arm’s 959,000 in the same active wall-clock — ten generations against roughly nine hundred — and importance weights formed over ten generations are expected to be poor whatever produced them. What the data support is the weaker conjunction: *at a fixed wall-clock budget* the barrier costs one to two-and-a-half orders of magnitude of throughput, and on this workload that deficit is large enough to cost the reported posterior as well. Separating the barrier’s own effect on the weights from the effect of the evaluations it forgoes needs a matched-evaluation-count twin, which we have not run (§9).

Is the twin’s fine generation doing the work? The twin above synchronizes every $W=16$ evaluations, one per worker — a finer generation than the synchronous baseline’s population of 100 — so one could suspect the ratio is an artifact of barrier *frequency* rather than of synchronization as such. We tested this directly by re-running the twin with the barrier coarsened to one per 112 evaluations ($7W$, matching the baseline’s population), five replicates at every slowdown (Table 3, third row). Granularity turns out to matter only where the straggler does not: at the $0\times$ and $1\times$ controls the coarse twin is $3.3\times$ and $2.3\times$ faster ($48.7 \rightarrow 162.8$ and $65.3 \rightarrow 148.4$ simulations/s), cutting the asynchronous lead from $80\times$ to $24\times$ and from $48\times$ to $21\times$; but at $5\times$, $10\times$ and $20\times$ the two granularities agree to three significant figures (31.6 vs 31.8 , 15.91 vs 15.94 , 7.98 vs 7.98), leaving the ratios at 102 – $402\times$. The mechanism is straightforward: a barrier costs either its own latency or the wait for the slowest member of the generation, whichever is larger. Coarsening amortizes the latency over more evaluations, which helps when latency dominates, but every generation still has to wait for the straggler no matter how long the generation is — so in the straggler regime the cost is granularity-independent. The advantage we report under a straggler is thus not an artifact of the twin’s schedule, and the remaining caveat is one of workload rather than granularity: this is a deliberately adversarial near-instant simulator, and where per-evaluation cost amortizes the coordination (§7.3) the barrier’s relative cost is correspondingly smaller.

The same test under runtime heterogeneity.

A persistent straggler is the sharpest case but not the typical one, so we repeat the twin comparison on the heterogeneity benchmark, where every worker draws a fresh lognormal runtime multiplier at every evaluation rather than one rank being permanently slow. Both barrier granularities again, five replicates at each of the five heterogeneity levels, simulation-limited with the budget per level sized from measured rates (Table 4). The picture differs from the straggler case in two instructive ways. First, the $\sigma=0$ control now costs essentially *nothing*: the twin runs at 44.8 simulations/s against the asynchronous arm’s 47.6, a factor of 1.1, where on the straggler benchmark the same control already cost 24 – $80\times$. The difference is the simulator, not the barrier — these evaluations take ≈ 1 s rather than ≈ 4 ms, so barrier latency is negligible against the work it separates. The barrier’s cost on a cost-bearing simulator is therefore a pure *variance* cost, and it appears only once there is variance to pay for: it grows monotonically to $10.1\times$ at $\sigma=1.5$ and $25.3\times$ at $\sigma=2$ at the baseline-matched granularity ($30.9\times$ at the finer one). Second, coarsening the barrier now *does* help — 1.2 to $1.7\times$ across $\sigma \geq 0.5$, against the $1.00\times$ it bought under the persistent straggler. That contrast is the mechanism made visible: coarsening trades a heavier per-barrier maximum against half as many barriers, which is a favourable trade when the slow draw is transient and a different worker each time, and no trade at all when the same rank is slow in every generation. Posterior quality is again unaffected — the twin recovers the analytic posterior mean to within 0.003 – 0.034 absolute error against the asynchronous arm’s 0.007 – 0.011 , with the largest twin value at $\sigma=2$, where its budget is only ten generations.

Table 3 Barrierized twin on the straggler benchmark: median simulation throughput (simulations per second of active wall-clock) over five replicates. The twin is the *same* propagator with a collective barrier before each breed, so the rows differ only in synchronization. Two barrier granularities are shown: one barrier per $W=16$ evaluations (one per worker, the finest possible generation) and one per 112 evaluations ($7W$, matching the synchronous baseline’s population of 100). Coarsening the generation to the baseline’s costs the twin nothing once the straggler binds — the $5\times/10\times/20\times$ columns agree to three significant figures — and buys a factor of 3.3/2.3 only at the $0\times/1\times$ controls, where barrier latency rather than straggler waiting is the bottleneck. The lower block reports posterior quality as W_1 between each arm’s *reported* estimator (the full-history AMIS posterior) and the *analytic* posterior. We do not report W_1 to a point mass at the true parameter here: the analytic posterior for this configuration has standard deviation 0.1, so an exactly correct answer scores 0.090 on that statistic and every arm scores 0.069–0.085 — it has no resolution on this benchmark, and it rewards an over-concentrated archive. Note that the arms are not at a matched evaluation budget (last block); see the discussion in §7.1.

Straggler slowdown	0×	1×	5×	10×	20×
Asynchronous (ours)	3888	3112	3241	3218	3205
Barrierized twin, every $W=16$	48.7	65.3	31.6	15.9	8.0
Barrierized twin, every 112	162.8	148.4	31.8	15.9	8.0
Ratio, twin every W	80×	48×	103×	202×	402×
Ratio, twin every 112	24×	21×	102×	202×	402×
<i>Reported posterior vs. analytic posterior, median W_1</i>					
Asynchronous (ours)	0.010	0.013	0.011	0.011	0.009
Barrierized twin, every $W=16$	0.011	0.008	0.015	0.737	1.729
Barrierized twin, every 112	0.012	0.011	0.009	0.016	1.067
<i>Evaluations completed (median); the arms are not budget-matched</i>					
Asynchronous (ours)	1166k	934k	972k	965k	959k
Barrierized twin (both)	75.1k	44.5k	12.3k	6.3k	3.2k

There is one further reading worth stating, because it runs opposite to the concern the twin was built to address. The worry about a cross-algorithm comparison is that it might *overstate* the barrier’s role. Here it understates it: the synchronous pyABC baseline’s penalty grows only from $2.1\times$ to $4.8\times$ over the same heterogeneity range, so at $\sigma=2$ our own barrierized twin is $5.3\times$ slower than the external baseline at matched barrier granularity. The design difference that accounts for it is the one noted in §9, item iv: pyABC closes a generation on the first 100 evaluations to finish and discards the latecomers, so it never waits for the slowest draw, whereas the twin — like our own propagator, minus asynchrony — keeps every evaluation and therefore pays the full maximum. The barrier’s unmitigated cost is thus larger than the measured gap over pyABC, and the headline comparison is conservative rather than flattering.

Table 4 Barrierized twin under *runtime heterogeneity* (each worker draws a fresh lognormal runtime multiplier of scale σ at every evaluation), 48 workers, median throughput over five replicates. Same two barrier granularities as Table 3: one per $W=48$ evaluations and one per 96, the latter matching the synchronous baseline’s population of 100. Unlike the straggler benchmark, the $\sigma=0$ control costs almost nothing ($1.1\times$) because these evaluations take ≈ 1 s and barrier latency is negligible against them — the cost here is purely the wait for the slowest draw, and it grows with σ . Coarsening the barrier helps under transient heterogeneity ($1.2\text{--}1.7\times$), where under a persistent straggler it bought nothing. The synchronous pyABC baseline degrades far less than our own twin because it closes a generation on the first 100 finishers and discards latecomers, so at $\sigma=2$ the twin is $5.3\times$ slower than the external baseline it stands in for. All rates are simulations per second of each arm’s own elapsed wall-clock; at $\sigma=1.5$ and 2 the baseline’s runs end at ≈ 54 and ≈ 52 s of the 60 s budget, so dividing by its actual elapsed time credits it with the higher rate and makes the comparison against it conservative.

Heterogeneity σ	0	0.5	1.0	1.5	2.0
Asynchronous (ours)	47.6	42.1	28.9	17.6	11.4
Barrierized twin, every $W=48$	44.8	15.1	4.37	1.06	0.37
Barrierized twin, every 96	45.1	19.8	6.50	1.75	0.45
Synchronous pyABC baseline	23.1	18.9	9.91	4.35	2.39
Ratio, twin every W	$1.1\times$	$2.8\times$	$6.6\times$	$16.6\times$	$30.9\times$
Ratio, twin every 96	$1.1\times$	$2.1\times$	$4.4\times$	$10.1\times$	$25.3\times$
Ratio, pyABC baseline	$2.1\times$	$2.2\times$	$2.9\times$	$4.1\times$	$4.8\times$

Runtime heterogeneity.

Under stochastic log-normal runtime noise the asynchronous method sustains higher utilization (lower idle fraction) than the synchronous baseline, which idles at generation barriers (Fig. 2). Crucially, this utilization converts directly into inference quality: at a fixed wall-clock budget the synchronous baseline completes fewer and fewer simulations as the runtime spread σ grows (it idles at every barrier) and its posterior-mean error to the analytic Gaussian posterior climbs several-fold (from ≈ 0.05 to ≈ 0.28), whereas the asynchronous method completes several-fold more simulations and holds its error essentially flat ($\approx 0.007\text{--}0.011$) (Fig. 3). Removing the barrier therefore preserves not just the simulation rate but the *inference* delivered per unit wall-clock.

Parameter-coupled runtime (every-particle bias).

The study above slows simulations independently of their parameters; a sharper test couples runtime to the *inferred parameter*, directly probing the every-particle bias of Limitation (iv). On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) we set the post-evaluation delay to $d(\mu) = \min(d_0 e^{c(\mu-\mu_c)/\ell}, d_{\max})$ with base $d_0 = 0.02$ s, centre $\mu_c = 0$, length scale $\ell = 0.15$, and cap $d_{\max} = 0.5$ s, and sweep the coupling strength $c \in \{0, 0.5, 1, 2\}$ ($c = 0$ recovers uniform runtime). Larger c makes

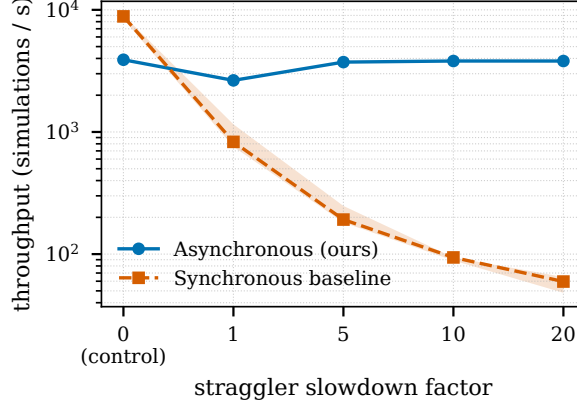


Fig. 1 Throughput (median + inter-quartile range over five replicates) versus straggler slowdown factor, including a $0\times$ no-straggler control. One worker carries a permanent post-evaluation delay of 0.1 s scaled by the factor. The synchronous baseline degrades sharply at the generation barrier (from ≈ 8800 sims/s at the control to ≈ 60 at $20\times$); the asynchronous method is robust (median ≈ 2600 – 3900 across the range, with no collapse). Note the logarithmic throughput axis.

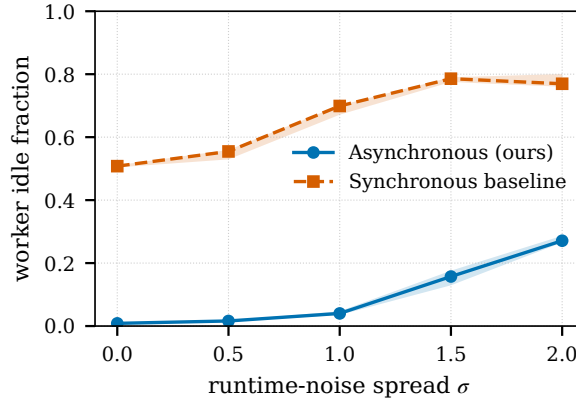


Fig. 2 Worker utilization loss under runtime heterogeneity: the fraction of worker wall-clock spent idle versus the runtime-noise spread σ (five replicates; bands are inter-quartile ranges). The synchronous baseline idles at generation barriers (its idle fraction rises from ≈ 0.51 to ≈ 0.79 as heterogeneity grows) while the asynchronous method stays near-fully utilized (idle fraction ≈ 0.01 rising to ≈ 0.27).

one region of parameter space evaluate far faster than its mirror image, so the asynchronous “keep every accepted particle” rule over-represents the fast region in the raw archive. The coupling has the intended effect: aggregate throughput falls as it steepens, by $\approx 43\%$ for the asynchronous method from its peak ($\approx 1880/2010 \rightarrow 1150$ sims/s; it is flat between $c = 0$ and $c = 0.5$ and falls monotonically thereafter) and monotonically and more steeply for the barrier-bound baseline ($\approx 840 \rightarrow 250$; Fig. 4a), confirming that the fast region dominates the raw sample counts. Yet the asynchronous posterior-mean error to the analytic Gaussian posterior does not increase as the coupling steepens; it

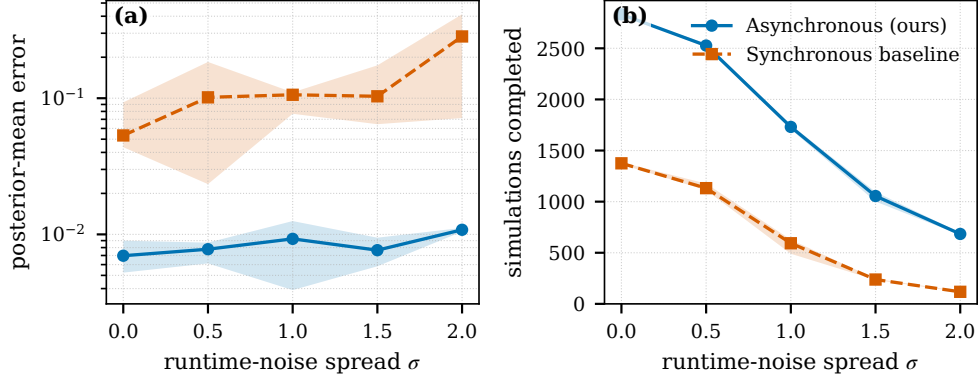


Fig. 3 Inferential efficiency under runtime heterogeneity (Gaussian-mean benchmark, analytic posterior, fixed 60 s budget, five replicates; bands are inter-quartile ranges). Left: posterior-mean error to the analytic posterior versus runtime-noise spread σ ; the synchronous baseline degrades sharply while the asynchronous method stays flat. Right: simulations completed within the budget; the synchronous baseline is starved by barrier idling as σ grows, whereas the asynchronous method sustains several-fold higher completion.

stays ≈ 0.01 and comparable to the discard-latecomers synchronous baseline (Fig. 4b). At the coupling levels tested the over-representation is measurable in the raw sampling rate but does not measurably distort the reported (unweighted top- k) posterior mean. A targeted ablation that removes the AMIS proposal adaptation ($S=0$) and re-runs the same coupling sweep leaves this robustness essentially unchanged — the asynchronous posterior-mean error stays ≈ 0.005 – 0.013 with or without AMIS and shows no trend in the coupling — which indicates that the raw archive is itself robust at these levels rather than that AMIS actively corrects the over-representation. Two finer tests that Limitation (iv) motivates sharpen this rather than leave it open. First, we score the AMIS-*reweighted* posterior — the estimator the guarantees of §4 concern, resampling the archive by the retroactive weight (8) — instead of the unweighted archive, and compare it against the no-AMIS variant ($S=0$) across the coupling sweep and a steeper regime (base delay 0.5 s, cap 8 s). The reweighted posterior recovers the analytic mean as accurately as the unweighted archive (absolute error ≤ 0.025 , no trend in the coupling) in both regimes and with or without AMIS; the reweighting’s effect surfaces in the posterior *spread*, not the mean — it lowers the effective sample size (ESS fraction ≈ 0.88 , vs ≈ 0.99 without AMIS), roughly triples the maximum normalised weight (≈ 0.03 vs ≈ 0.01), and widens the central 90% interval by $\approx 15\%$ — so AMIS reshapes the tails but neither creates nor removes a mean bias at these coupling levels. Second, a drain-after-deadline variant lets every in-flight simulation finish past the wall-clock deadline and recomputes the posterior; comparing it against the censored run isolates the completion-time censoring that dividing by the proposal density does not correct. Draining moves the weighted posterior mean by at most 0.013 — comparable to the replicate-to-replicate spread (≈ 0.01) — with no systematic direction, in both the mild and the steep regime. At these coupling levels the deadline censoring is

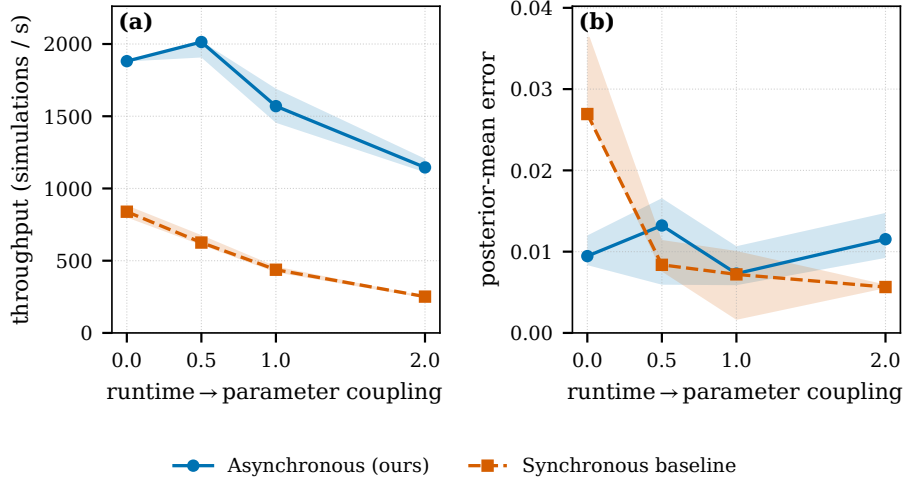


Fig. 4 Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens — monotonically for the barrier-bound baseline, and from a peak at $c = 0.5$ for the asynchronous method — confirming that the fast region dominates the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous method and the discard-latecomers synchronous baseline, so the over-representation does not measurably distort the reported posterior at these coupling levels.

therefore present in principle but negligible in effect, consistent with the sampler concentrating in the fast region near the posterior where few simulations are in flight at the deadline. We accordingly report the raw archive as robust here rather than claiming AMIS corrects the over-representation, and treat a formal censoring correction as future work (Limitation (iv)).

Strong scaling.

Lotka–Volterra is deliberately an *adversarial* simulator for a scaling study: each evaluation takes only ≈ 2 ms, so the simulator occupies only a small fraction of each worker’s wall-clock and there is essentially no per-evaluation compute to distribute. A dedicated instrumented run measures this productive fraction directly (Fig. 6): at the 48-worker throughput peak a worker spends only $\approx 25\%$ of its time inside the simulator, leaving $\approx 75\%$ for coordination and idle waiting, and that productive share falls to $\approx 8\%$ and then $\approx 2\%$ as the job spreads from one node to three to six. The mechanism is the method’s single shared population: every completed evaluation is broadcast to all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. On a near-instantaneous simulator this exchange saturates and then actively slows the productive ranks. Aggregate throughput therefore rises almost linearly while the job fits on one node ($149 \rightarrow 369 \rightarrow 1983 \rightarrow 6240$ sims/s at $1 \rightarrow 4 \rightarrow 16 \rightarrow 48$ workers) and then *declines* across fully packed multi-node jobs (worker counts that are exact multiples

Table 5 Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 5, left, shows the inter-quartile spread). Asynchronous (ours) at both archive sizes vs. synchronous baseline. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 the synchronous baseline is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous method peaks on one node and then declines as multi-node coordination dominates, so the baseline overtakes it beyond three nodes at $k = 100$. Enlarging the archive tenfold costs throughput at every scale — the per-arrival proposal reconstruction is $\mathcal{O}(k)$ — and moves that crossover in to a single node.

Workers	1	4	16	48	144	192	240	288
Nodes	<1	<1	<1	1	3	4	5	6
Synchronous	105	96	437	2952	3757	4322	4539	6034
Asynchronous ($k = 100$)	149	369	1983	6240	4195	2615	2088	1729
Asynchronous ($k = 1000$)	106	133	620	3804	2804	1957	1587	1336

of 48): $6240 \rightarrow 4195 \rightarrow 2615 \rightarrow 2088 \rightarrow 1729$ sims/s at $48 \rightarrow 144 \rightarrow 192 \rightarrow 240 \rightarrow 288$ workers, i.e. $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ nodes, because adding ranks beyond one node lengthens the all-to-all exchange faster than it adds useful work. This is a property of coordinating a near-instantaneous simulator across a single shared population, not of asynchrony as such (precisely the regime in which asynchrony has least to offer), and it lifts once the simulator carries real per-evaluation cost that amortizes the coordination (the Cellular Potts result of §7.3; Fig. 5, right). To keep the comparison fair at high core counts we give the synchronous ABC-SMC baseline a *population size equal to the worker count* (a smaller population cannot occupy every core); it then uses all allocated cores and scales steadily, from ≈ 440 sims/s at 16 workers to ≈ 6000 at 288 (Table 5). On this free simulator the asynchronous method leads while the job stays within a few nodes ($\approx 4.5\times$ at 16 workers, $\approx 2\times$ at one full node), but its lead *erodes* with scale and the fair synchronous baseline overtakes it beyond three nodes, where the growing all-to-all coordination cost dominates and the asynchronous method leaves most of the allocation idle. This is the honest boundary of the asynchronous advantage on a cheap, homogeneous simulator; it is largest exactly in the heterogeneous and straggler-prone regimes where barriers otherwise waste compute. The archive size sets where that boundary falls, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$: enlarging the archive tenfold to $k = 1000$ costs throughput at every worker count — 39% at the single-node peak ($6240 \rightarrow 3804$ sims/s) — and brings the crossover with the fair baseline in from beyond three nodes to a single node (Table 5). Sweeping k across $\{50, 100, 200, 400, 800, 1000\}$ at every worker count resolves that cost into a smooth monotone decline in throughput and a single step in the crossover, which moves in from beyond three nodes to three nodes at $k \geq 400$ (Table 7). The archive size is therefore a systems cost as well as a statistical knob — but, put on the same axis as the calibration it buys, the two turn out *not* to trade off over this range: the best-calibrated archive is also within 7% of the fastest, and every archive larger than $k = 100$ is both slower and worse calibrated (§7.2, Table 7).

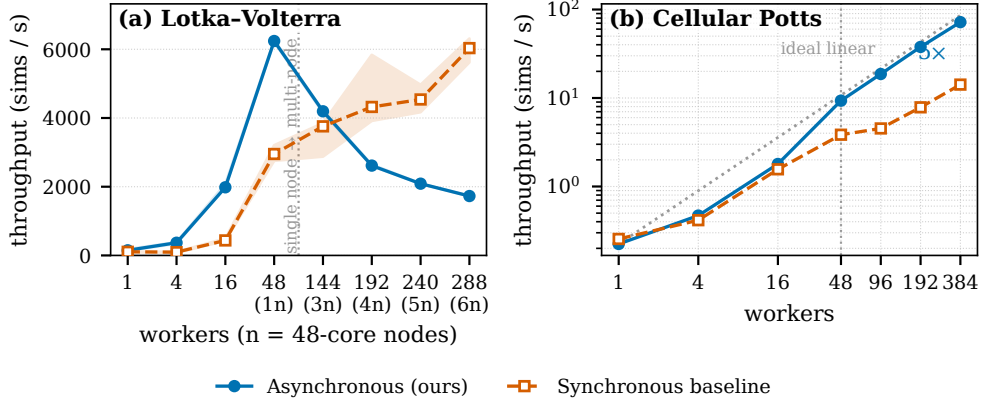


Fig. 5 Strong scaling: median simulation throughput versus worker count at a fixed wall-clock budget, asynchronous (ours) vs. a synchronous baseline given a population of $\max(100, W)$ so it can occupy every core: pyABC caps concurrency at the population size, so at $W \leq 100$ the default $k=100$ already exceeds the worker count and is fair as it stands, and above one full node the baseline is re-run at $k=W$. A “ W -worker” point gives both arms W cores, but the baseline’s MPI sampler spends its root rank coordinating rather than simulating, so it has $W-1$ simulating ranks against the asynchronous method’s W — at most a 2.1% handicap at $W=48$, falling to 0.3% at $W=384$, i.e. shrinking exactly where the reported gap grows. **Left:** Lotka–Volterra (near-instant simulator, ≈ 2 ms/eval; 180 s budget; categorical x -axis; bands are inter-quartile ranges over five replicates; node count n in labels). With no per-evaluation compute to distribute, the asynchronous method’s single-population all-to-all coordination cost grows with the worker count: throughput rises within one node, peaks there, and then declines across multi-node jobs, so its lead over the fair, steadily scaling baseline erodes and the baseline overtakes it beyond three nodes. **Right:** Cellular Potts (costly `cellsInSilico` simulator; 1800 s budget; log-log; medians over replicates). Real per-evaluation cost amortizes the coordination, so the asynchronous method tracks ideal-linear scaling (dotted) to eight nodes (384 workers, $\approx 97\%$ parallel efficiency) while the barrier-bound baseline scales only sub-linearly, a $\approx 5\times$ gap at 384 workers that *grows* with scale.

7.2 Posterior validity (RQ1)

Having established the systems advantage, we ask what it costs in inference quality. Across calibration and matched-budget posterior recovery the reported streaming posterior — the full-history AMIS estimator (8) — is comparable to the matched synchronous baseline in low dimensions and under multimodality, and in four dimensions errs toward conservatism (it mildly over-covers) rather than over-confidence.

Calibration.

Table 8 reports SBC empirical coverage on the Gaussian-mean model (1000 trials, matched simulation budget), for the reported full-history AMIS posterior. The asynchronous method tracks the nominal level closely (0.51/0.80/0.90/0.94 at the 0.50/0.80/0.90/0.95 levels), within Monte Carlo error of nominal at every level; the synchronous baseline under-covers at every level (0.35/0.63/0.75/0.83), i.e. it is over-confident. At 1000 trials a coverage estimate carries a Monte Carlo standard error of only $\approx \pm 0.007$ – 0.016 , well below the ≈ 0.12 – 0.17 gap between the methods, so the ordering is resolvable rather than noise: on this benchmark the streaming estimator

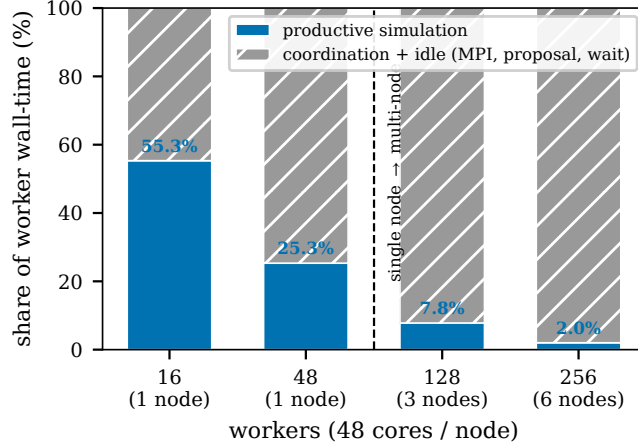


Fig. 6 Measured decomposition of asynchronous worker wall-time on Lotka-Volterra ($k = 100$; 180 s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated) — the share of worker wall-clock spent inside the simulator — collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate exactly as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 5 is therefore a measured property of coordinating a near-instantaneous simulator, not an inferred one.

is *better* calibrated than the matched synchronous baseline. Reporting the estimator over the full evaluated history rather than the top- k archive alone matters even here: in a paired 500-trial run that changes only the reported support, the truncated archive mildly under-covers at the top levels (0.89/0.92 at the 0.90/0.95 levels), which the full-history report corrects to nominal — a reporting-support effect that grows with dimension, as the multimodal and four-dimensional tests below make explicit. This test is on a one-dimensional model with an analytic posterior; the higher-dimensional and multimodal settings where the archive/proposal approximations are most likely to bite are probed separately below. Figure 7 shows the corresponding coverage curve and rank histogram.

Calibration under multimodality and in higher dimensions.

To probe exactly those settings, we run SBC (1000 trials, matched simulation budget) on two harder targets, reporting the same full-history AMIS posterior. The first is a *multimodal* model $y_i \sim \mathcal{N}(\theta^2, \sigma^2)$: the sign ambiguity gives a symmetric bimodal posterior at $\pm\sqrt{y}$, the case a top- k archive is most at risk of collapsing onto a single mode. It does not collapse — the archive retains *both* modes in 85.7% of trials and the mode containing the true value in 86.7% (the remainder are largely near- $\theta=0$ trials where the two modes merge), the importance weights stay near-uniform (median effective-sample-size fraction 0.99, median maximum normalised weight 0.011), and coverage tracks the nominal level (0.50/0.82/0.91/0.96), calibrated within Monte Carlo error. The second is the *four-dimensional* g-and-k model, where the reporting-support choice bites. The reported full-history posterior is *conservative*: it mildly over-covers at every level and parameter (e.g. empirical coverage 0.62/0.91/0.96/0.99 at the

0.50/0.80/0.90/0.95 levels for the location parameter A , and similarly for B, g, k), because the finite- ϵ smooth-ABC posterior it reports is over-dispersed at the loose bandwidth a four-dimensional budget reaches. The efficient top- k archive used for online proposal sits at the opposite end: it is over-concentrated and would instead *under-cover* (0.85/0.89 at the 0.90/0.95 levels for A), with healthy weights throughout (median effective-sample-size fraction 0.81), so the miscalibration is a reporting-support effect, not weight degeneracy. The calibrated posterior is therefore *bracketed* between the over-concentrated archive core and the over-dispersed full history, and the amount of history reported is the dispersion knob: widening the reported support from the archive toward the full history raises coverage monotonically and continuously through nominal — for A , $0.85/0.89 \rightarrow 0.93/0.96 \rightarrow 0.95/0.98 \rightarrow 0.97/0.99$ at the 0.90/0.95 levels for reported sizes $M = 200/500/1500/\text{full}$, crossing nominal near $M \approx 400\text{--}800$ (the shape parameters B and k are already near-nominal at $M=500$, e.g. 0.89/0.94 for B). This is a property of *how much of the history is reported*, not of the AMIS weighting (whose importance weights stay well-conditioned) or of the budget: doubling the simulation budget to 80,000 per trial leaves the archive coverage essentially unchanged (0.85/0.89 at the 0.90/0.95 levels for A , within Monte Carlo error for B, g, k). We accordingly report the streaming estimator, evaluated over the full evaluated history, as well calibrated in one dimension and under multimodality, and conservative — over-covering rather than over-confident — in four dimensions; an intermediate reported support ($M \approx 400\text{--}800$) or a larger archive tighten the four-dimensional report toward exactly nominal.

How calibration depends on the archive size and the snapshot buffer.

The tests above fix k and S per benchmark, which leaves open whether the two efficiency knobs must be retuned as the dimension grows. To settle this we sweep them directly on a d -dimensional Gaussian-mean target, $y_i \sim \mathcal{N}(\theta, \sigma^2 I_d)$ with $\theta \in \mathbb{R}^d$ under a flat box prior, whose posterior is known exactly at every d (SBC against that exact posterior returns 0.496/0.794/0.905/0.952, confirming the target itself is calibrated, so any departure below is a property of the method). The grid is $d \in \{1, 2, 4, 8, 16\} \times k \in \{50, 100, 200, 400, 800\} \times S \in \{0, 5, 20, 50\}$, 1000 trials per cell, reporting the same full-history estimator (Table 6).

The two knobs behave quite differently. The snapshot buffer S has a threshold and then saturates: at $S = 0$ the estimator is over-confident in low dimensions (mean deviation -0.088 at $d = 1, 2$), $S = 5$ removes essentially all of it (-0.001), and $S = 20$ and $S = 50$ add nothing further — at any dimension. The value of a richer balance-heuristic denominator therefore does *not* grow with d over this range, and the fixed $S = 20$ we use throughout sits comfortably above the knee. The archive size k is the opposite: its effect is large, strongly dimension-dependent, and *not* monotone. Small archives can be severely over-confident in intermediate dimensions — at $d = 8, k = 50$ empirical coverage is 0.29/0.52/0.63/0.71 against the nominal 0.50/0.80/0.90/0.95, a failure that persists at every S — while intermediate archives over-cover in the same regime ($+0.18$ to $+0.20$ at $k = 200\text{--}400$). We note that this does not follow the floor a naive count of covariance parameters would predict: the perturbation covariance is a full $d \times d$ matrix estimated from the archive, suggesting $k \gtrsim d(d+1)/2$, yet $d = 8$

fails at $k = 50 > 36$ while $d = 16$ remains within 0.06 of nominal at $k = 50 < 136$. The reading this supports is that $d = 16$ is protected by its own difficulty: with 20,000 simulations over a $[-5, 5]^{16}$ prior the sampler barely concentrates, so the reported posterior stays broad and covers almost by default, whereas $d = 8$ has enough signal to concentrate but not enough to concentrate correctly. Two further runs test that reading rather than leaving it as narrative. First, it predicts that a *higher* dimension should look benign again: at $d = 32$ the coverage is 0.55–0.57, 0.83–0.84, 0.92, 0.96 at the four levels for every $k \in \{50, 100, 800\}$ — mildly conservative and nowhere near the $d = 8$ failure, even at $k = 50$. Second, the failure is not a budget short-fall: doubling the simulation budget at $d = 8, k = 50$ makes coverage slightly *worse* (0.26/0.47/0.58/0.67 against 0.29/0.52/0.63/0.71 at the original budget), exactly as expected if the extra simulations sharpen an already mislocated posterior rather than repairing it. We therefore do not offer a closed-form rule for k ; the practical guidance is that the archive size is the dispersion knob (§7.2), that its safe range is problem- and dimension-specific, and that it should be chosen by checking calibration directly rather than by a formula.

The archive size does not, in this range, trade calibration against throughput.

Because k has both a statistical effect (above) and a systems cost — per-arrival proposal reconstruction is $\mathcal{O}(k)$, so a larger archive is slower at every worker count (§7.1) — it is natural to expect the two to pull against each other, and to have to buy calibration with throughput. Measuring both on one axis shows that expectation is wrong over the range we can test. Table 7 puts the throughput cost of k next to its worst-case miscalibration over the dimensions swept, using a throughput grid matched on everything but k (Lotka–Volterra, 180 s wall cap, five replicates, worker counts 1–288: $k \in \{100, 1000\}$ from the main scaling run and $k \in \{50, 200, 400, 800\}$ from a dedicated k -frontier run sharing that configuration). Of the six archive sizes, only two are Pareto-optimal, and they are the two smallest: $k = 100$ is simultaneously the best-calibrated setting tested (worst deviation 0.064, at $d = 4$) and within 7% of the fastest (6240 vs 6684 simulations/s at the single-node peak). Every larger archive is dominated by it on *both* axes at once — $k = 200$ is 5% slower and nearly three times worse calibrated (0.180), $k = 400$ is 14% slower and 0.195, and $k = 800$ costs 31% of throughput to end up still slightly worse calibrated than $k = 100$ (0.074). The only genuinely faster archive, $k = 50$, is the catastrophic one (0.250 at $d = 8$). So the practical shape of the k decision is not a frontier to negotiate but a floor to clear: go far enough above the small-archive failure region, and then stop, because further enlargement buys no calibration and costs throughput monotonically.

Two qualifications keep this from being read too broadly. The join is on k alone and the two axes come from different benchmarks by design — calibration needs a target whose posterior is known at every d , throughput needs the regime where an $\mathcal{O}(k)$ per-arrival cost is most exposed. That second choice makes the throughput column an *upper* bound on the relative cost of k : Lotka–Volterra evaluations take ≈ 2 ms, so coordination dominates there, and on a cost-bearing simulator the per-evaluation work amortizes the archive operation (§7.3). The conclusion is therefore conservative

Table 6 Calibration sensitivity to the archive size k and the AMIS snapshot buffer S on the d -dimensional Gaussian-mean target (1000 trials per cell; entries are the mean signed deviation of empirical from nominal coverage over the 0.50/0.80/0.90/0.95 levels and the d parameters). Negative is over-confident, positive is conservative; 0.000 is exactly calibrated. **Left:** varying k at $S = 20$. **Right:** varying S at $k = 100$.

	archive size k (at $S = 20$)				
	50	100	200	400	800
$d = 1$	-0.014	-0.003	+0.006	+0.006	+0.074
$d = 2$	-0.020	-0.006	+0.009	+0.030	+0.072
$d = 4$	+0.025	+0.064	+0.094	+0.133	+0.063
$d = 8$	-0.250	+0.055	+0.180	+0.195	+0.061
$d = 16$	-0.047	+0.028	+0.049	+0.053	+0.055

	snapshot buffer S (at $k = 100$)			
	0	5	20	50
$d = 1$	-0.088	-0.001	-0.003	-0.005
$d = 2$	-0.088	-0.001	-0.006	-0.003
$d = 4$	+0.039	+0.068	+0.064	+0.064
$d = 8$	+0.065	+0.006	+0.055	+0.074
$d = 16$	-0.010	+0.010	+0.028	+0.034

in the right direction — even where k is most expensive, the best-calibrated archive is nearly free. What k *does* change at scale is where the fair synchronous baseline overtakes the asynchronous method, and there it acts as a step rather than a smooth trade: the crossover sits beyond three fully packed nodes for $k \leq 200$ and moves in to three nodes for $k \geq 400$ (last column).

Archive concentration.

We first report the top- k concentration diagnostic, on all three benchmarks and for completeness with the earlier literature, and then turn to the estimator the paper reports. Read as what it is — distance from the archive to a point mass, which rewards concentration whether or not the posterior is right (§6.2) — it says the following. On g-and-k the asynchronous method concentrates slightly *tighter* than the synchronous baseline at matched wall-clock budget (0.078 vs. 0.090, inter-quartile ranges [0.077, 0.083] and [0.082, 0.093]), and on Lotka–Volterra the two are comparable with the asynchronous method marginally tighter (0.316 vs. 0.332, overlapping inter-quartile ranges); on Lotka–Volterra both numbers rest on the $\approx 2\%$ of simulations that do not go extinct (§6.2). On the expensive Cellular Potts simulator the ordering reverses and the asynchronous method is *modestly but consistently behind*: the async–sync difference is positive across 87.5% of the budget, ending at +0.043 with an inter-quartile envelope of [0.019, 0.052] that excludes zero, and it never exceeds ≈ 0.06 against a Wasserstein scale of ≈ 0.4 (Figs. 8c and 9). One property of this benchmark

Table 7 The archive size on both of its axes at once. **Throughput:** median asynchronous throughput at the single-node peak (48 workers) on Lotka–Volterra, 180 s wall cap, five replicates ($k \in \{100, 1000\}$ from the main scaling run, $k \in \{50, 200, 400, 800\}$ from the matched k -frontier run), and its change relative to the $k=100$ we use throughout. **Calibration:** worst absolute deviation of empirical from nominal coverage over the dimensions of the d -dimensional Gaussian-mean sweep at $S=20$ (Table 6), with the dimension attaining it. **Crossover:** fully packed nodes at which the fair synchronous baseline (population = worker count) overtakes the asynchronous arm. Only $k=50$ and $k=100$ are Pareto-optimal; every larger archive is both slower *and* worse calibrated than $k=100$. The sweep did not include $k=1000$.

k	sims/s at $W=48$	vs. $k=100$	worst $ \Delta\text{cov} $	at	baseline overtakes
50	6684	+7%	0.250	$d=8$	beyond 3 nodes
100	6240	—	0.064	$d=4$	beyond 3 nodes
200	5904	−5%	0.180	$d=8$	beyond 3 nodes
400	5393	−14%	0.195	$d=8$	3 nodes
800	4331	−31%	0.074	$d=1$	3 nodes
1000	3804	−39%	—	—	3 nodes

Table 8 SBC empirical coverage on Gaussian-mean inference (parameter μ , 1000 trials; Monte Carlo standard error $\approx \pm 0.007$ – 0.016), for the reported full-history AMIS posterior. Closer to the nominal level is better.

Method	0.50	0.80	0.90	0.95
Synchronous baseline	0.35	0.63	0.75	0.83
Asynchronous (ours)	0.51	0.80	0.90	0.94

bounds how that gap should be read: the Wasserstein-to-reference *rises* with compute for both methods (asynchronous $0.37 \rightarrow 0.48$), because the Cellular Potts parameters are only weakly identified (§7.3) and the comparison is against a reference point rather than a known posterior, so a diffuse cloud scores better than a posterior that has concentrated slightly off that reference. The metric therefore penalizes concentration on this target, and the asynchronous method — which completes $\approx 2\times$ the evaluations in the same budget — concentrates fastest. Per-parameter posteriors and joint marginals are consistent with the known posteriors on the benchmarks that have one, and on Cellular Potts they are consistent for one parameter of two: the motility reference sits inside the central 90% of both methods’ marginals (at their 87th and 91st percentiles), whereas the division-rate reference falls below *every* one of the 500 reported draws for the asynchronous method and for the synchronous baseline alike (5th percentiles 0.19 and 0.21 against a reference of 0.05). Both methods therefore exclude the division-rate reference outright rather than merely failing to resolve it, and rejection ABC places it only at its own 5th percentile while remaining nearly uniform. This is the identifiability property of the benchmark asserting itself and it is common to all three methods

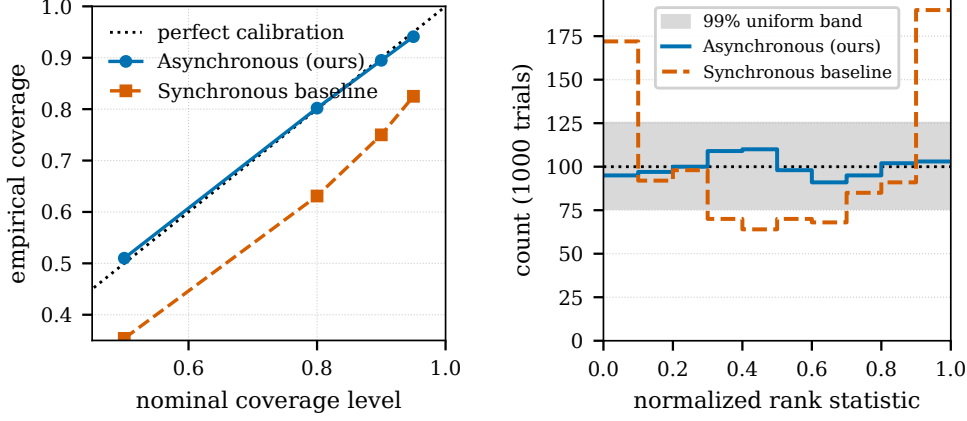


Fig. 7 Simulation-based calibration on Gaussian-mean inference (1000 trials; the reported full-history AMIS posterior): empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.

(§7.3); it is the reason no quality claim on Cellular Potts should be read as a bound on attainable accuracy. On the Gaussian-mean benchmark the asynchronous method recovers the posterior mean to within ≈ 0.002 – 0.038 absolute error, comparable to the synchronous baseline’s ≈ 0.006 – 0.021 and far tighter than a rejection-ABC reference; the SBC calibration above is unaffected.

Scoring the reported estimator against a reference posterior.

The curves above measure the top- k archive against a point mass at the true parameter, which is a concentration diagnostic and not posterior quality (§6.2). On the two benchmarks that admit a reference posterior we can do better and score the estimator this paper actually reports — the retroactive AMIS posterior of (8), the object Theorem 1 is about — against that reference. We do it over time by replaying `extract_posterior` on the prefix of the history that had completed by each checkpoint, which is what the run would have reported had it stopped there; the synchronous baseline is scored on its last completed generation and rejection ABC on its accepted set, so each method is judged on its own reported object (Fig. 10). The reference is the analytic posterior for Gaussian-mean, and for g-and-k the posterior given the same seven octile summaries the discrepancy is built from — *not* the full-data posterior, which would measure the summaries’ information loss rather than the sampler (Appendix C).

Two things come out of it, and the second was not what we went looking for.

First, the estimator recovers the posterior, and the two competitive methods are indistinguishable. On Gaussian-mean the reported AMIS posterior sits at $W_1 = 0.008$ – 0.016 from the analytic posterior across the 300s budget, against 2.35 for rejection ABC. This is the claim we make for the streaming estimator, and it is a claim the point-mass metric was incapable of supporting: on this configuration that metric floors at 0.090, which is what an *exactly correct* posterior scores, and every method sat at or below it (§7.1).

Second, the effective sample size does not grow with the budget. The reported posterior’s ESS is 237–297 particles at every checkpoint of the Gaussian run while the evaluated history grows from 1.0×10^5 to 1.3×10^6 ; the run’s own stored weights give 261–303 across the five replicates. Thirteen times the compute buys no additional effective particles, which is also why the accuracy curve above is flat rather than improving. The mechanism is not asynchrony — it reproduces in a serial driver — and it is not a defect of the replay. It is the interaction between where the estimator is reported and how the bandwidth is chosen: the posterior is reported at the tightest bandwidth the run reached, that bandwidth is set by ESS-retention bisection against the archive and therefore tracks the archive downwards, and so the set of particles carrying non-negligible kernel weight stays at a few multiples of k while everything outside it is weighted to near zero. A controlled sweep makes the scaling explicit: over $k \in \{50, 100, 200\}$ and $n \in \{5 \times 10^3, 2 \times 10^4, 8 \times 10^4\}$, ESS/k stays within 1.9–3.6 at all nine points while ESS/n falls like $1/n$. The full history enters the mixture denominator; it does not, in any effective sense, enter the numerator.

We think this is the single most useful thing to know about the method, and it reframes three results we had reported separately. It is why reporting over the full history rather than the top- k archive helps calibration but only up to a point, and why an intermediate reported support of $M \approx 400$ –800 was where coverage crossed nominal: that is 4 – $8k$, about where the effective support already sits. It is consistent with the measured failure of the CLT’s rate condition for the reported configuration (§4.1): an asymptotic variance that shrinks at rate n is not available from an effective sample that does not grow with n . And it says plainly where the method’s remaining headroom is — in k and in the bandwidth rule, not in more workers. The throughput results of §7.1 stand on their own as systems results; what they buy in posterior accuracy, on a benchmark this well specified, is bounded by an effective support that more simulation alone does not enlarge.

7.3 Realistic simulator: Cellular Potts (RQ3)

The Cellular Potts benchmark runs end-to-end with the real `cellsInSilico` simulator, a realistic, expensive, heterogeneous-runtime workload and the regime the method targets.

Strong scaling.

Because each Cellular Potts evaluation carries real cost, the per-arrival coordination that capped Lotka–Volterra is amortized, and the asynchronous method scales almost linearly all the way to eight nodes (Fig. 5, right): median throughput rises from 0.2 to 72.0 simulations/s over 1 to 384 workers and, unlike the near-instantaneous Lotka–Volterra case, keeps scaling *across* the single-node→multi-node boundary, each node-doubling roughly doubling throughput ($9.3 \rightarrow 18.7 \rightarrow 37.8 \rightarrow 72.0$ sims/s at $48 \rightarrow 96 \rightarrow 192 \rightarrow 384$ workers), for $\approx 97\%$ parallel efficiency relative to the single-node rate. The synchronous baseline, given a population equal to the worker count so it too can occupy every core, scales only *sub-linearly* over the same range ($3.8 \rightarrow 4.5 \rightarrow 7.8 \rightarrow 14.2$ sims/s at $48 \rightarrow 96 \rightarrow 192 \rightarrow 384$ workers, $\approx 50\%$ efficiency), so at 384 workers the asynchronous method sustains $\approx 5\times$ its throughput, a gap that *widens*

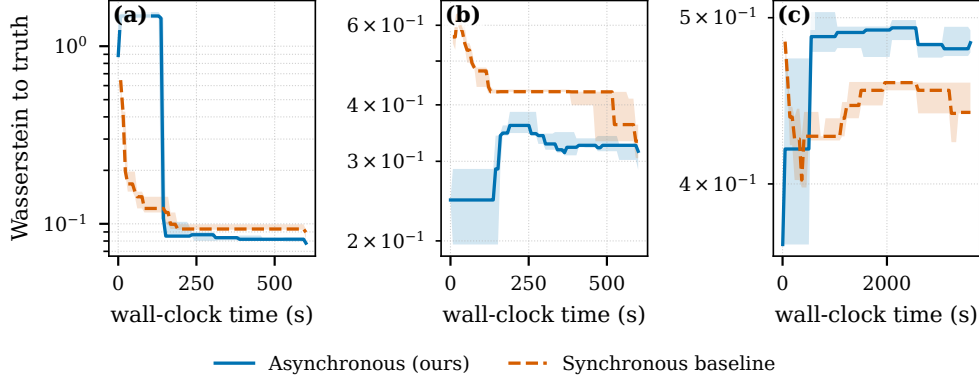


Fig. 8 Posterior quality (Wasserstein distance to the truth/reference, lower is better) versus wall-clock time on g-and-k (left), Lotka–Volterra (middle), and Cellular Potts (right), asynchronous vs. synchronous; shaded bands are inter-quartile ranges over five replicates. On g-and-k the asynchronous method reaches a slightly lower Wasserstein distance within the budget, on Lotka–Volterra the two are comparable, and on Cellular Potts the asynchronous method is modestly but consistently behind. Note that on the weakly identified Cellular Potts target the distance-to-reference *increases* with compute for both methods, so the metric penalizes the method that concentrates fastest (§7.2).

with scale. This is the regime the method is built for: an expensive, heterogeneous simulator where the generation barrier, not per-arrival overhead, is the binding cost. Worker utilization makes the mechanism explicit (Fig. 11): the asynchronous method keeps workers ≈ 98.9 – 99.8% busy across all scales, whereas the synchronous baseline, even with every core filled within a generation, then waits for the slowest simulation at the barrier, so its utilization falls to ≈ 24 – 29% ($41\% \rightarrow 24\% \rightarrow 26\% \rightarrow 29\%$ from 48 to 384 workers, i.e. ≈ 70 – 76% idle at every multi-node scale). This barrier idle, not per-arrival overhead, is what holds the synchronous throughput to sub-linear scaling while the asynchronous method tracks the ideal. (We report medians throughout.)

The barrierized twin at this scale.

The twin comparison of §7.1 isolates synchronization on the two synthetic benchmarks; here we run it on the real simulator, at the same four worker counts and the same $k=100$, simulation-limited at 40 evaluations per worker (Table 9). Two things come out of it, and the second is the more interesting.

First, the barrier alone costs a factor that is remarkably *stable* across scale: $1.9\times$ at one, two and four nodes, rising to $2.8\times$ at eight. The external pyABC baseline’s penalty over the same range grows much faster, from $2.4\times$ to $5.1\times$. The twin is therefore consistently the faster of the two synchronous arms, and the distance between them widens with scale — which is the honest answer to the question the twin was built to ask, and it is not the flattering one. On this workload the barrier is a *major but not exclusive* cause of the cross-algorithm gap: it accounts for roughly half to three-quarters of it in log terms, and the residual, which grows with worker count, has to come from pyABC’s other differences (its probabilistic-rejection step, its classical

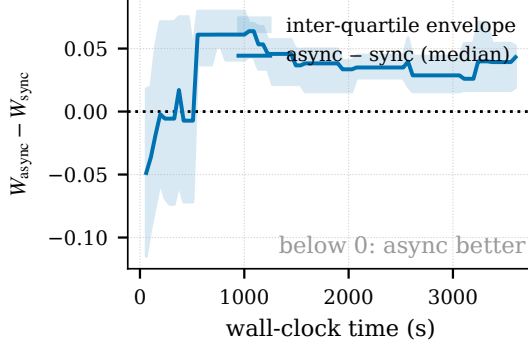


Fig. 9 Cellular Potts posterior-quality *difference* (asynchronous minus synchronous Wasserstein to the reference) versus wall-clock time, with a zero reference line and an inter-quartile envelope over five replicates (the per-method Cellular Potts curves of Fig. 8c, differenced). The difference is positive — asynchronous behind — across 87.5% of the budget, ending at +0.043 with an envelope of $[0.019, 0.052]$ that excludes zero; it straddles zero only over the first ≈ 510 s. The gap is nonetheless small in absolute terms, never exceeding ≈ 0.06 against an absolute Wasserstein of ≈ 0.4 , and it is measured against a reference point on a weakly identified target whose distance-to-reference grows as either method concentrates (§7.2, §7.3).

population weighting, and its own coordination costs) rather than from synchronization. We therefore do *not* claim the Cellular Potts gap is attributable to the barrier alone — only that the barrier is the single largest identified component of it. This is also the opposite ordering to the straggler and heterogeneity benchmarks, where the twin ran far *slower* than pyABC because pyABC’s latecomer discarding rescues it from waiting on the slowest draw; with no injected pathology, that rule has little to exploit and our more efficient propagator keeps the twin ahead.

Second — and this is the more useful finding — the barrier does not merely reduce throughput, it makes throughput *unpredictable*. The asynchronous arm’s five replicates agree to within 1.3% at every scale (coefficient of variation 0.6–1.3%), while the twin’s spread across the *same* seeds is 25–54%: at 48 workers its replicates run at 1.7, 2.1, 4.9, 5.9 and 6.0 simulations/s, a factor of 3.5 between fastest and slowest. The mechanism is the one the utilization measurement already shows: a generation ends only when its slowest evaluation does, so the whole job inherits the cost of the most expensive parameter draw in each generation, and on this benchmark simulation cost varies strongly with the parameters. Asynchrony removes that coupling — a slow draw delays only its own worker — which is why the asynchronous arm’s throughput is reproducible to within a percent while its barrierized twin, running identical proposals from identical seeds, varies by more than threefold. For a practitioner this is arguably the more consequential of the two effects: a median slowdown can be budgeted for, a factor-of-three variance in time-to-result cannot.

One note on the twin as an instrument. It intermittently deadlocks at ≥ 192 ranks — we observed five hangs while collecting this table, one in the MPI teardown and the others mid-run — so individual replicates had to be retried. Every combination did complete, and all twenty entries here are full runs; but the twin is a diagnostic built

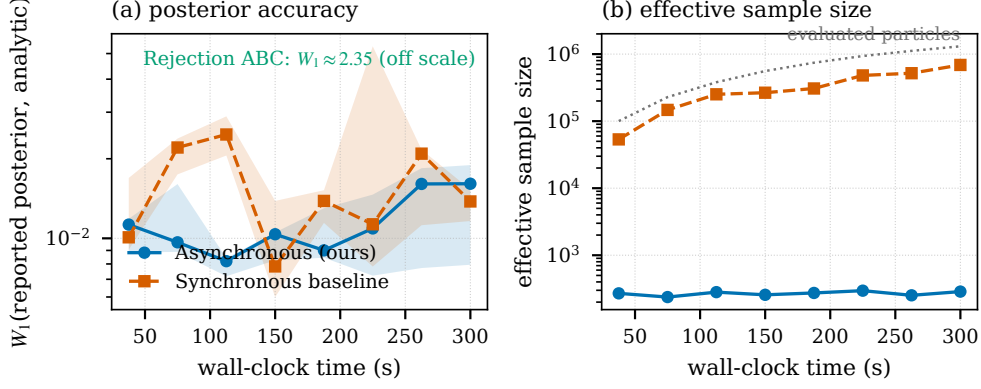


Fig. 10 What the *reported* estimator recovers, and with how many effective particles. Each method is scored on the posterior it actually reports — for the asynchronous method the retroactive AMIS estimator replayed over the prefix completed by each checkpoint, for the synchronous baseline its last completed generation, for rejection ABC its accepted set — against a reference posterior rather than a point mass at the truth. **Top:** W_1 to the reference, median and inter-quartile range over five replicates. **Bottom:** the effective sample size of that same reported posterior, with the evaluated-particle count (dotted) on the same axis. The gap between the two lines is the point: the history grows by more than an order of magnitude and the effective sample does not follow it.

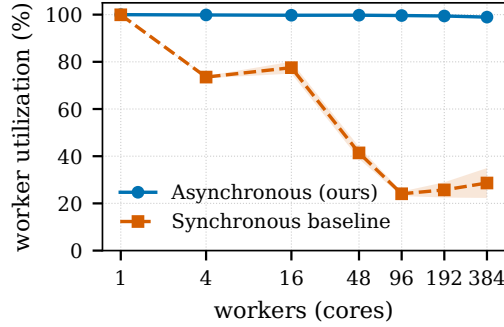


Fig. 11 Cellular Potts worker utilization (fraction of worker wall-clock spent inside the simulator) versus worker count (1800 s budget; error bars are ± 1 s.d.); the synchronous baseline uses a population equal to the worker count (Fig. 5, right). The asynchronous method stays ≈ 98.9 – 99.8% utilized at every scale, while the synchronous baseline, even filling every core within a generation, then idles at the barrier waiting for the slowest simulation, so its utilization falls to ≈ 24 – 29% ($41\% \rightarrow 24\% \rightarrow 26\% \rightarrow 29\%$ at 48/96/192/384 workers, i.e. ≈ 70 – 76% idle at every multi-node scale). This barrier idle is the measured mechanism behind the synchronous method’s sub-linear scaling in Fig. 5 (right).

for this comparison rather than a robust implementation, and at these rank counts it needs supervision.

Posterior quality.

On the inference side the asynchronous method produces marginals of similar shape to the synchronous baseline (Fig. 12), though it trails modestly on distance-to-reference

Table 9 Barrierized twin on the Cellular Potts workload: median throughput (simulations/s) over five replicates, $k=100$, simulation-limited at 40 evaluations per worker. CV is the coefficient of variation across replicates. The barrier’s own cost is nearly flat from one to four nodes ($1.9\times$) and rises to $2.8\times$ at eight, while the external pyABC baseline’s penalty grows from $2.4\times$ to $5.1\times$ over the same range — so the barrier is the single largest identified component of the cross-algorithm gap but not the whole of it, and the residual grows with scale. The second row-pair is the more useful finding: the twin’s replicate spread is 25–54% against the asynchronous arm’s 0.6–1.3% on the same seeds, because a generation inherits the cost of its most expensive parameter draw. Synchronization makes throughput unpredictable as well as lower. The pyABC baseline has three replicates at 192 and 384 workers (its fair population = W shards); all twin and asynchronous entries have five.

Workers (= cores)	48 (1 node)	96 (2)	192 (4)	384 (8)
Asynchronous (ours)	9.33	18.67	37.75	71.98
CV	0.9%	1.3%	0.6%	1.3%
Barrierized twin	4.87	10.00	20.24	25.54
CV	50.7%	53.8%	24.8%	49.8%
Synchronous pyABC baseline	3.85	4.53	7.85	14.17
Ratio, twin	$1.9\times$	$1.9\times$	$1.9\times$	$2.8\times$
Ratio, pyABC baseline	$2.4\times$	$4.1\times$	$4.8\times$	$5.1\times$

(§7.2): the cell-motility parameter concentrates near its reference value, while the division rate is not merely unresolved but mislocated — every method, the baseline and rejection ABC included, places its mass away from the reference (§7.2). That is a property of the configuration rather than an incidental finding: this benchmark was built to supply a realistic heterogeneous-runtime workload for the scaling and throughput measurements, and its summaries, discrepancy metric and parameter ranges were never selected for identifiability, with the two parameters confounding. Quality comparisons on it should be read as evidence that no method is advantaged by the setup, not as a measurement of what any of them can achieve. This weak identification is a structural property of the inference problem, not of asynchrony: at this scale the division-rate and motility parameters are near-degenerate (both chiefly modulate the cell count that the summaries resolve), so neither method separates them and both improve only modestly on rejection ABC. The matched comparison therefore isolates the systems advantage as largely *orthogonal* to identifiability: both methods are limited chiefly by the problem. On the summary metric the asynchronous method is modestly behind over the full budget (+0.043 in Wasserstein-to-reference against a scale of ≈ 0.4 ; §7.2), so we report its throughput advantage as bought at a small quality cost on this benchmark rather than at none — with the caveat that the metric rewards dispersion on a weakly identified target, and that the asynchronous method, concentrating fastest, is the one it penalizes most. Establishing the same quality with a synchronous matched baseline at the still larger scales the method targets would itself be prohibitively wasteful, precisely the cost the barrier-free design removes.

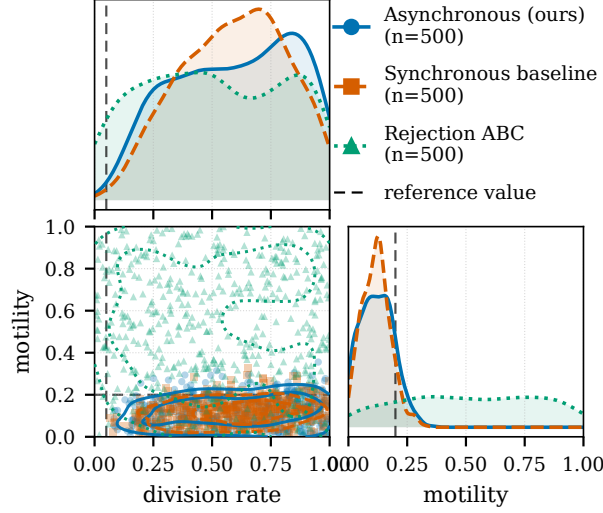


Fig. 12 Cellular Potts posterior (division rate, motility) on the real `cellsInSilico` simulator: asynchronous (blue), synchronous baseline (red), and rejection ABC (green), with dashed lines at the reference values. The asynchronous posterior is drawn from its AMIS estimator (500 resampled draws); the synchronous baseline and rejection ABC show their final populations (500 each). The asynchronous method and the synchronous baseline both place their motility mass at low values, bracketing the motility reference (which sits at the 87th and 91st percentiles of their marginals). Neither recovers the division rate: its reference lies below every one of the 500 draws for both, and rejection ABC puts it at its own 5th percentile — the summaries and parameter ranges of this benchmark were never selected for identifiability (§7.3), so the panel bounds no method’s attainable accuracy. Both do concentrate more tightly than rejection ABC.

7.4 Ablation

Reverting the smooth kernel to a hard indicator slightly raises the Wasserstein-to-truth relative to the full method (though within its confidence interval), while removing the AMIS cumulative-mixture reweighting is approximately neutral on this uniform-runtime analytic target (Fig. 13). On the Gaussian-mean benchmark the full method attains a final Wasserstein-to-truth of 0.073 (CI [0.067, 0.079]); the hard-indicator kernel raises it to 0.076 (inside that interval), whereas dropping AMIS leaves it essentially unchanged (0.072; Fig. 13a), with the remaining hyperparameter variants scattered around the full-method reference and degrading only toward the largest perturbation scales (0.084–0.087). That AMIS is neutral here is consistent with the coupling study (§7.1), where removing AMIS likewise left the posterior essentially unchanged: with no runtime–parameter coupling on this uniform-runtime target, there is no overrepresentation for the reweighting to reshape. The AMIS-isolation trajectory (Fig. 13b) accordingly shows the full method and the no-AMIS variant tracking each other throughout the converged regime (both ≈ 0.072 – 0.073 after 40 s).

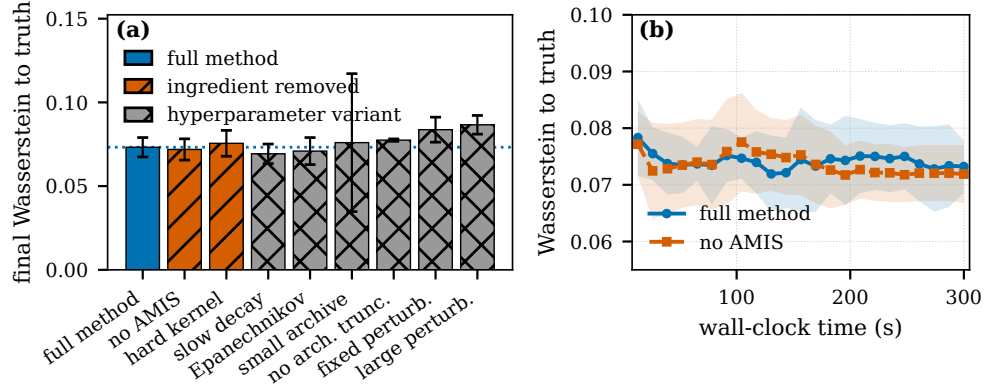


Fig. 13 Ablation on the Gaussian-mean benchmark (analytic posterior; 48 workers, 300 s budget, $k = 100$, five replicates, asynchronous method). (a) Final Wasserstein-to-truth for the full method (blue, solid), the two ingredient removals (vermillion, hatched: no AMIS, hard kernel), and hyperparameter variants (grey, cross-hatched); colour and hatch encode the class redundantly, error bars are 95% CIs, and the dotted line marks the full-method mean. (b) AMIS isolation: full method vs. no-AMIS on a shared wall-clock grid (bands are ± 1 s.d. over replicates), zoomed to the converged regime; on this uniform-runtime analytic target the two variants track each other.

7.5 Sensitivity

Across the full hyperparameter grid *on the Gaussian-mean benchmark* (the analytic-posterior sanity model) the asynchronous method’s posterior quality is stable, with no region of catastrophic failure (Fig. 14). The Wasserstein distance to the true parameter stays within ≈ 0.09 – 0.15 across the grid (each cell averaged over five replicates and the two smooth kernels); the archive size k matters only marginally *for point quality at this dimension* (its effect on calibration in higher dimensions is a separate and much larger story, Table 6), and quality degrades only mildly toward the largest perturbation scales and initial tolerances. The tolerance scheduler is inert under a smooth kernel — the asynchronous selector chooses ϵ by ESS-retention bisection regardless of the scheduler type — so we fixed it to the acceptance-rate rule rather than sweeping a variable that cannot change the smooth-kernel result. We therefore did not find delicate tuning necessary to reach the quality reported above *on this benchmark*. We did not repeat the full grid on the costlier non-Gaussian benchmarks, so this robustness statement is scoped to the analytic-posterior model and to the qualitative trends observed there (scheduler-insensitive; mild, monotone sensitivity to perturbation scale and initial tolerance) rather than asserted as a general guarantee. Two further scope limits matter. This grid measures *point quality* (Wasserstein to the truth) in *one* dimension; it says nothing about calibration, and the archive size that looks inconsequential here is decisive for coverage in higher dimensions — the dimension-resolved sweep of §7.2 (Table 6) finds a genuinely catastrophic region ($d = 8$, $k = 50$: coverage 0.29 at the 0.50 level) that a one-dimensional quality grid cannot see. “No failure region” should therefore be read as a statement about one-dimensional point quality only.

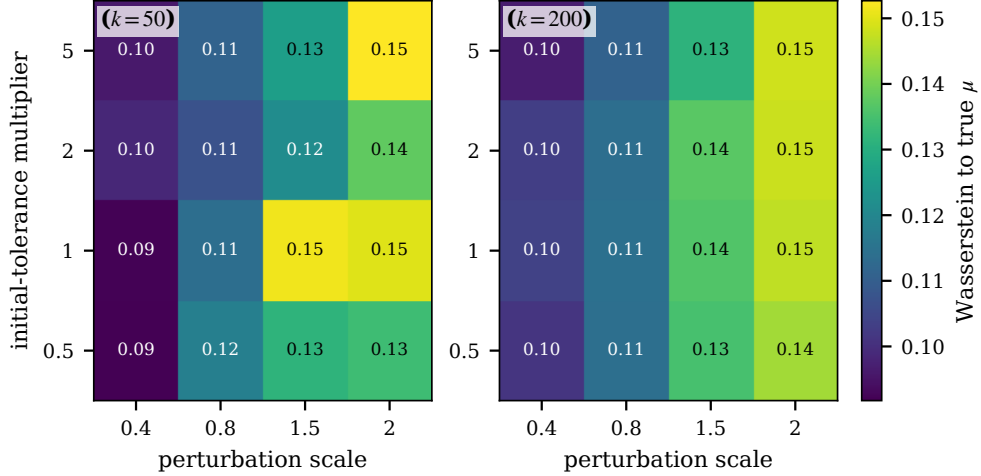


Fig. 14 Hyperparameter sensitivity of the asynchronous method, measured by posterior quality (Wasserstein distance to the true μ , lower is better; averaged over replicates and smoothing kernel). Each panel sweeps perturbation scale (x) against the initial-tolerance multiplier (y); panels are faceted by archive size k . Quality is stable across the grid (Wasserstein ≈ 0.09 – 0.15) with no failure region; the tolerance scheduler is inert under the smooth kernel and is held fixed, while larger perturbation scales degrade quality only mildly.

8 Discussion

The results support the three claims. Asynchronous execution helps most where it should: under a persistent straggler and under runtime heterogeneity, the generation barrier is the dominant cost and removing it preserves throughput that the synchronous baseline loses. For homogeneous, near-instantaneous simulators at large worker counts the asynchronous coordination overhead does not pay off: the multi-node Lotka–Volterra points, where the fair synchronous baseline overtakes the asynchronous method, make this honest boundary explicit. On the realistic Cellular Potts workload (the cost-bearing regime the design targets) the method scales almost linearly while the synchronous baseline, held at the generation barrier even when given a matching population, scales only sub-linearly — at a small measured cost in posterior quality on that benchmark (§7.2), on a metric that penalizes the concentration the extra throughput buys.

9 Limitations

We are explicit about what §4 does and does not give. (i) AMIS estimators are consistent but not finite-sample unbiased, since proposals depend on past particles; a local-decision rule in the spirit of [19] that recovers finite-sample unbiasedness for ABC is future work. (ii) The bandwidth floor $\epsilon_\infty > 0$ in Assumption 5 means the theorems characterize the smooth-ABC posterior at the limit bandwidth, not the exact posterior at $\epsilon = 0$; and for the central limit theorem (though not for consistency) that limit

must be *deterministic*, since the centred integrand it is built on would otherwise vary with the run. Setting `min_tol` bounds the limit below but only a schedule that reaches its floor makes it deterministic. The reported runs leave `min_tol` unset and use the data-driven schedule, so neither holds, and the bandwidth a finite budget reaches is not an asymptotic limit at all. (iii) The theorems hold up to the limiting ratio r of (9) between the density the sample was drawn from and the function the estimator divides by; $r \equiv 1$ gives the smooth-ABC posterior exactly, and §4.1 decomposes r into four factors and *measures two of them* — the prior floor and the snapshot quadrature — at $\hat{\zeta} = 0.022$ on the Cellular Potts production run, a 1.1% shift in total variation against a large- m reference. The other two, the truncation constant and the capped-rejection/redraw conditioning, are bounded rather than measured, and because the factors multiply that is a partial accounting rather than a bound on the whole. We do not prove $r \equiv 1$. Under asynchronous execution one of the four factors additionally absorbs the difference between the emitted proposal density and the conditional law under a filtration the proofs can use (Assumption 4), which is a property of the schedule rather than of the estimator. The stabilization clause (Assumption 5(ii)) is not merely unproved for the top- k archive rule but measurably violated in the configuration we run: the proposal’s distance to its limit decays far too slowly (§4.1), so the central limit theorem covers the damped- or frozen-adaptation variants rather than these experiments, while consistency, which needs no rate, is unaffected. (iv) Keeping every accepted particle (unlike pyABC’s discard-latecomers rule) over-represents fast-simulator parameter regions in the raw archive. The parameter-coupled-runtime study (§7.1, Fig. 4) quantifies this: steepening the coupling cuts throughput as the fast region comes to dominate the raw counts, yet — across the sweep, a steeper regime, and with or without AMIS — neither the unweighted posterior mean nor the AMIS-reweighted posterior’s mean drifts with the coupling (AMIS reshapes the tails, not the mean), and a drain-after-deadline test finds the completion-time censoring negligible in effect at these levels (numbers there). We therefore do not correct for it algorithmically and do not claim the AMIS reweighting cancels it: dividing by the proposal density corrects proposal adaptation, not selection by completion time, so a formal correction would need completion probabilities or an anytime-sampling argument, and a residual finite-sample effect at stronger couplings cannot be ruled out — both left to future work. (v) The snapshot buffer is fixed at $S = 20$; adaptive sizing is a natural extension. (vi) Simulation-based calibration of the reported posterior — the full-history AMIS estimator (8) — shows it well calibrated in one dimension and robust to multimodality (retaining both modes of a bimodal target in $\approx 86\%$ of trials), and conservative rather than over-confident on the four-dimensional g-and-k model, where it mildly *over-covers*. This four-dimensional behaviour is a reporting-support effect, not a weighting or budget failure (§7.2): the efficient top- k archive used for online proposal is over-concentrated and would under-cover, the full evaluated history is over-dispersed and over-covers, and coverage rises monotonically through nominal as the reported support grows from one to the other, so the calibrated posterior is bracketed between the two. We report the conservative full-history end by default; an intermediate reported support or a larger archive would tighten the four-dimensional

report to nominal. The denominator’s prior floor acts in the same direction for a different reason, and the two are worth not conflating: the floor suppresses the far tails of the reported posterior (§4.1(a)) and so tightens it, whereas widening the reported support adds tail mass. Both are small next to the effect measured here — the floor and the snapshot quadrature together move the reported posterior by about one percent in total variation (§4.1). (vi-b) The reported posterior’s effective sample size is set by the archive size and not by the simulation budget: $\text{ESS}/k \in [1.9, 3.6]$ across a sweep of k and n , and 237–303 effective particles out of $1.1\text{--}1.3 \times 10^6$ evaluated on the production Gaussian run (§7.2). Barrier-free execution therefore delivers more *evaluations* per second, which is what §7.1 measures, but converting those into posterior accuracy is limited by a reporting rule we did not design for that purpose. Raising k raises the effective sample proportionally and is the obvious lever; a bandwidth rule that does not track the archive downwards, or reporting at a deliberately looser ϵ than the tightest reached, are the others. We have not swept any of them for this purpose, and until that is done the honest scope of the systems result is throughput rather than time-to-accuracy. (vii) The synchronous baseline is matched on the ABC kernel and bandwidth schedule, but it still differs from our method in more than the barrier: it adds a probabilistic-rejection step, its generation length depends on the resulting acceptance probability, and it retains classical population weighting rather than the streaming estimator. Matching the kernel removes the acceptance *rule* as a confound, and the *barrierized twin* of §7.1 (Table 3) closes the remaining gap on the straggler benchmark: the same propagator with a collective barrier before each breed runs 21–402 $\times$ slower than the asynchronous arm, so on that workload the throughput gain is attributable to synchronization rather than merely dominated by it. The twin does *not* isolate synchronization for the posterior comparison, only for the throughput one: a collective barrier needs identical per-rank call counts, so the twin cannot be stopped on wall-clock and the two arms end at budgets that differ by two orders of magnitude (§7.1). A matched-evaluation-count twin, which would separate the barrier’s effect on the importance weights from the effect of the evaluations it forgoes, is the obvious next control and we have not run it. The granularity objection — that the twin synchronizes every W evaluations, a finer generation than the baseline’s population of 100 — is measured rather than assumed away: re-running the twin with the barrier coarsened to the baseline’s population changes its throughput by less than 1% at every slowdown from $5\times$ up, and matters (2.3–3.3 $\times$) only at the controls where barrier latency and not straggler waiting is the bottleneck (Table 3). The twin has since been repeated under runtime heterogeneity (Table 4), where it costs $1.1\times$ at the $\sigma=0$ control and $25.3\times$ at $\sigma=2$ at the baseline-matched granularity, so the attribution now covers both synthetic HPC pathologies rather than only the sharpest one. That test also shows the cross-algorithm comparison to be conservative in the direction that matters: pyABC’s own penalty grows only to $4.8\times$, because it discards latecomers instead of waiting for the slowest draw (item iv), so the barrier’s unmitigated cost exceeds the gap we report against it. The twin has since also been run at scale on the Cellular Potts workload (Table 9), and there the answer is more qualified. The barrier costs $1.9\times$ from one to four nodes and $2.8\times$ at eight, while pyABC’s penalty over the same range grows from $2.4\times$ to $5.1\times$: the barrier is the largest single component we can identify, but it is not

the whole gap, and the residual — pyABC’s probabilistic-rejection step, its classical population weighting, its own coordination — grows with worker count. On the realistic simulator we therefore attribute the gain to synchronization only in part, and say so rather than generalizing the straggler result. The attribution now covers all three benchmarks on which we report systems gains, with the caveat that its strength differs across them: near-total under a straggler or heterogeneity, partial on Cellular Potts. Separately, the twin is a diagnostic rather than a robust implementation — it intermittently deadlocks at ≥ 192 ranks (five hangs while collecting Table 9), so replicates there needed retrying, though all twenty entries are complete runs. Separately, on a methodological note for reproducibility: the post-run retroactive estimator builds its denominator from $m \leq S+1 = 21$ history-reconstructed proposals (with a defensive prior floor), not the full evaluated history; for a near-instantaneous simulator whose history reaches many millions of particles the pass is evaluated in bounded-memory chunks, which changes memory, not the estimate.

10 Conclusion

We presented a generation-free, single-arrival-driven ABC algorithm that combines smooth-kernel ABC with streaming AMIS reweighting, reports an estimator that is a pure function of the evaluated history and is therefore recoverable by replay, and integrates into Propulate’s barrier-free island model. The estimator it reports is consistent, and satisfies a central limit theorem with an explicit asymptotic variance under rate conditions we measure rather than assume — finding that the configuration reported here does not meet them, so the central limit theorem covers damped-adaptation variants; both are proved directly rather than inherited, under conditions relating its denominator to the density its sample was drawn from, which we state precisely, decompose, and partly measure on production runs — but do not establish for the configuration the experiments use. The reported posterior is validated empirically by simulation-based calibration — well calibrated in low dimensions and under multimodality, and conservative rather than over-confident in higher dimensions, where we trace the residual miscalibration to how much of the evaluated history is reported (the efficient top- k proposal would under-cover; the full history over-covers) rather than to the weighting. Against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule, and scored on each method’s own reported posterior against a reference posterior rather than on a point mass at the truth, the two are indistinguishable on the benchmarks where such a reference exists, while the method delivers substantial HPC benefits on heterogeneous and straggler-prone workloads, including a realistic Cellular Potts simulation.

That comparison also turned up the result we would most want a reader to carry away, because it bounds the first one. The reported posterior’s effective sample size is set by the archive size and not by the simulation budget: 237–303 effective particles out of $1.1\text{--}1.3 \times 10^6$ evaluated, flat across a thirteen-fold growth of the history within a single run, with ESS/k between 1.9 and 3.6 across a controlled sweep of both. Reporting at the tightest bandwidth reached, while that bandwidth is chosen by ESS-retention bisection against the archive, leaves an effective support of a few multiples of k no

matter how many particles are evaluated. Removing the barrier therefore buys evaluations, reliably and by one to two-and-a-half orders of magnitude on the workloads we test; converting those evaluations into posterior accuracy is a separate problem, and on this evidence it is the one that now limits the method. We think generation-free ABC is the right execution model for large-scale simulator-based inference on modern HPC systems, and that the next work on it belongs in the reporting rule rather than in the scheduler.

Acknowledgements. *To be added in the final version.*

Declarations

Funding

To be added in the final version.

Conflict of interest/Competing interests

The authors declare no competing interests.

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The experiment outputs backing every figure and table (raw per-particle records, per-rank phase-timing logs, and the throughput/utilization/coverage summary tables), together with the resolved run configurations, will be deposited in a public repository upon publication.

Materials availability

Not applicable.

Code availability

The full implementation (the Propulate propagator, the matched-kernel pyABC baseline, and the analysis and plotting scripts that regenerate every figure and table from the deposited outputs) will be released in a public repository upon publication.

Author contribution

To be added in the final version.

Appendix A Proofs

A.1 Setting

We work with a filtration $(\mathcal{F}_i)$ for which, writing $\mathbb{E}_{i-1}[\cdot] = \mathbb{E}[\cdot \mid \mathcal{F}_{i-1}]$, the pair $\xi_i = (\theta_i, \rho_i)$ is $\mathcal{F}_i$ -measurable and the two conditional laws below hold. Particles are indexed by *proposal* time, not arrival time. This is what makes the martingale structure survive asynchronous execution: a worker forms its proposal from the history it has received at that moment, which is part of $\mathcal{F}_{i-1}$, so the conditional density $\tilde{q}_i$ of the emitted candidate is $\mathcal{F}_{i-1}$ -measurable and

$$\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i, \quad \rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i). \quad (\text{A1})$$

One further mismatch deserves naming, since it is between the theory and the software rather than inside either. The proofs index by *proposal* time, whereas the implementation's post-hoc estimator consumes a rank's *arrival*-ordered log and reconstructs the q_{τ_s} from prefixes of it. The two orderings agree for a sequential run and need not agree otherwise, so for an asynchronous run the object the code computes is the estimator of (8) evaluated on a different, equally valid history; we do not prove that the theorems transfer between the two, and the discrepancy is one more contributor to r . Under sequential execution $\mathcal{F}_i = \sigma(\xi_1, \dots, \xi_i)$ serves, and (A1) holds by construction. Under *asynchronous* execution it is a genuine assumption, and worth stating precisely what is being assumed. A discrepancy ρ_i may still be in flight when candidate $i + 1$ is proposed, so the σ -field of everything *realized* by that moment does not contain ξ_i , and Lemma 7 and the martingale arrays cannot use it. One must instead take a filtration large enough to contain the in-flight outcomes. Assumption 4 asserts that such a filtration exists *with $\tilde{q}_i$ still the conditional law of the candidate under it*. That is the substantive clause: a worker forms its proposal from the history it has received, so the proposal density is measurable with respect to the smaller field, but conditioning on the larger one need not leave the candidate's law unchanged when completion times steer which rank proposes next. When it does change, $\tilde{q}_i$ should be read as the enlarged-field conditional law rather than as the propagator's emitted density, and (9) is a statement about that object. This is the runtime-dependent censoring of §9, item (iv), seen from the theory side, and we do not prove it away. Concurrency also affects the reconstruction of the q_{τ_s} from a single rank's arrival-ordered log, which enters only through $\|\bar{q}_n - \bar{q}_\infty\|_\infty$ in Assumption 5.

No structural form is imposed on $\tilde{q}_i$: it is whatever the archive rule, the capped rejection, the underflow redraws and the bootstrap phase jointly produce, and it enters the proofs only through $\bar{q}_n^* = \frac{1}{n} \sum_{i \leq n} \tilde{q}_i$ of (6), for which the reported denominator $\bar{q}_n$ of (7) is an m -component stand-in. Let $\mathcal{Q}$ be the set of positive functions of the form (7) whose components satisfy Assumption 2; nothing below requires an element of $\mathcal{Q}$ to integrate to one. Elements of $\mathcal{Q}$ are parameterized by $p_q = (\text{means, component weights, covariances}, \alpha)$ ranging over a compact set $P_q \subset \mathbb{R}^D$, $D = m(kd + k + d(d + 1)/2) + m$.

Lemma 6 (bounded weights). *Under Assumptions 2–3, $0 \leq W_{i,n} \leq 1/\delta$ for all $i \leq n$ and all n , surely — whatever the sampler does.*

Proof All components of (7) are non-negative, so $\bar{q}_n \geq \delta\pi$ pointwise on Θ . Hence $W_{i,n} = \pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i) \leq K_{\epsilon_n}(\rho_i)/\delta \leq 1/\delta$, the last step by $K_\epsilon \leq 1$ (Assumption 1). The prior cancels exactly, so the bound needs neither a density-ratio condition nor a moment condition. $\square$

Lemma 7 (uniform SLLN for bounded martingale differences). *Let $P \subset \mathbb{R}^D$ be compact and $\{\psi_p\}_{p \in P}$ measurable functions on a measurable space $\mathcal{X}$ with $\sup_p \|\psi_p\|_\infty \leq B$ and $|\psi_p(x) - \psi_{p'}(x)| \leq \Lambda\|p - p'\|$ for every $x \in \mathcal{X}$. Let (ξ_i) be $\mathcal{X}$ -valued with ξ_i*

$\mathcal{F}_i$ -measurable. Then

$$\sup_{p \in P} \left| \frac{1}{n} \sum_{i \leq n} (\psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)) \right| \longrightarrow 0 \quad a.s.$$

Proof Fix p . The variables $D_i^p := \psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)$ form a martingale difference sequence with $|D_i^p| \leq 2B$, so by the Azuma–Hoeffding inequality $\mathbb{P}(|\sum_{i \leq n} D_i^p| \geq nt) \leq 2 \exp(-nt^2/8B^2)$ for every $t > 0$. Fix $\eta > 0$ and let P_η be a finite η -net of P , of cardinality at most $C_P \eta^{-D}$. A union bound gives $\mathbb{P}(\max_{p \in P_\eta} |\sum_{i \leq n} D_i^p| \geq nt) \leq 2C_P \eta^{-D} \exp(-nt^2/8B^2)$, which is summable in n (the net is fixed, so its cardinality does not grow with n); by Borel–Cantelli, $\limsup_n \max_{p \in P_\eta} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t$ a.s. For arbitrary $p \in P$ pick $p' \in P_\eta$ with $\|p - p'\| \leq \eta$; then $|D_i^p - D_i^{p'}| \leq 2\Lambda\eta$ by the Lipschitz hypothesis (which also bounds the conditional expectations), so $\limsup_n \sup_{p \in P} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t + 2\Lambda\eta$ a.s. Intersecting the almost-sure events over the countable family $t = 1/r$, $\eta = 1/j$ with $r, j \in \mathbb{N}$, gives the claim. $\square$

Lemma 8 (conditional-mean identity). *Let $\phi : \Theta \rightarrow \mathbb{R}$ be bounded measurable, let $\ell \in \{1, 2\}$ (an exponent, not the fidelity ratio r), and let $\epsilon \in [a, \epsilon_0]$ for some $a > 0$ and the positive function $q \geq \delta\pi$ be either deterministic or $\mathcal{F}_{i-1}$ -measurable. Then under (A1), writing $L_\epsilon^{(1)} := L_\epsilon$,*

$$\mathbb{E}_{i-1} \left[\frac{\phi(\theta_i) K_\epsilon(\rho_i)^\ell}{q(\theta_i)^\ell} \right] = \int_\Theta \phi L_\epsilon^{(\ell)} \frac{\tilde{q}_i}{q^\ell} d\theta.$$

Proof Condition on $\mathcal{F}_{i-1}$, given which ϵ and q are constants, and integrate first over $\rho_i \sim p(\cdot \mid \theta_i)$, which gives the factor $L_\epsilon^{(\ell)}(\theta_i) = \mathbb{E}[K_\epsilon(\rho)^\ell \mid \theta_i]$, then over $\theta_i \sim \tilde{q}_i$. The $\mathcal{F}_{i-1}$ -measurable case is what Steps 2 and 3 of the proof of Theorem 3 need; an $\mathcal{F}_\infty$ -measurable q — such as the limit $\bar{q}_\infty$ — would *not* be admissible here, which is why that proof centres on predictable proxies rather than on the limit. Fubini applies because the integrand is bounded by $\|\phi\|_\infty / (\delta\pi_-)^\ell$. The case $\ell = 1$, $\phi = \pi h$ is the one used for first moments; $\ell = 2$, $\phi = \pi^2 h^2$ gives the conditional second moments of Step 3(b) and Corollary 4. $\square$

The class used below.

Take $\mathcal{X} = \Theta \times [0, \infty)$, $\xi_i = (\theta_i, \rho_i)$, and, for a fixed bounded measurable h ,

$$\psi_{(\epsilon, p_q)}(\theta, \rho) := \frac{\pi(\theta) h(\theta) K_\epsilon(\rho)}{q_{p_q}(\theta)}, \quad (\epsilon, p_q) \in P_a := [a, \epsilon_0] \times P_q, \quad a \in (0, \epsilon_0]. \quad (\text{A2})$$

Each P_a is compact *and deterministic*, which matters: Lemma 7 cannot be applied over a parameter set that depends on the run, and $[\epsilon_\infty, \epsilon_0]$ would be such a set when ϵ_∞ is random. We therefore apply the lemma on $P_{1/j}$ for every $j \in \mathbb{N}$ with $1/j \leq \epsilon_0$ and intersect the countably many almost-sure events; since $\epsilon_n \downarrow \epsilon_\infty > 0$ a.s., each path has $\epsilon_n \in [1/j, \epsilon_0]$ for all n once $1/j \leq \epsilon_\infty$, so the resulting uniform bound covers $(\epsilon_n, \bar{q}_n)$ on that path. No deterministic bandwidth floor is required. On P_a , $\sup \|\psi\|_\infty \leq$

$\|h\|_\infty/\delta$ by the computation of Lemma 6. For the Lipschitz hypothesis: each truncated component density $\varphi(\theta; \mu_j, \Sigma)/Z(\mu_j, \Sigma)$ has parameter-derivatives bounded uniformly over $\theta \in \Theta$, because Θ is compact, $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ bounds φ and its derivatives, and the truncation mass is bounded below, $Z(\mu_j, \Sigma) \geq c_0 > 0$, uniformly for $\mu_j \in \Theta$. Here it matters that the implementation's constant is the *product of per-axis* in-box masses, not the exact joint mass: each factor is $\Phi((\text{hi}_d - \mu_{jd})/\sigma_d) - \Phi((\text{lo}_d - \mu_{jd})/\sigma_d)$, and since μ_{jd} lies in $[\text{lo}_d, \text{hi}_d]$ that interval contains a half-width $w_d/2$ on one side of μ_{jd} , giving a factor at least $\Phi(w_d/2\sigma_+) - \frac{1}{2} > 0$; the product over coordinates, $\prod_d (\Phi(w_d/2\sigma_+) - \frac{1}{2})$, is therefore bounded below by a constant depending only on the box and σ_+ , and is smooth in (μ_j, Σ) with bounded derivatives there. This is where Assumption 2's box hypothesis is used; compactness of Θ alone would not give it. Hence every $q \in \mathcal{Q}$ — a set of positive *functions*, not necessarily normalized (Assumption 2) — obeys $\delta\pi_- \leq q \leq q_+ < \infty$ and $q \mapsto q(\theta)$ is Lipschitz in p_q uniformly in θ ; so is $1/q$, since q is bounded away from 0. With K_ϵ Lipschitz in ϵ (Assumption 1) and πh bounded, (A2) satisfies the hypotheses of Lemma 7. Note that on $\{1/j \leq \epsilon_\infty\}$ we have $(\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n , and that $\bar{q}_\infty \geq \delta\pi$ by taking uniform limits.

A.2 Proof of Theorem 1

Work on the event $\{1/j \leq \epsilon_\infty\}$ for a fixed j , so that $\hat{p}_n := (\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n ; the conclusion then follows on the union over j , which has probability one. Fix a bounded measurable h and define, for each *fixed* $p \in P_{1/j}$, $\Gamma_i(p) := \mathbb{E}_{i-1}[\psi_p(\xi_i)]$ and $\bar{\Gamma}_n(p) := \frac{1}{n} \sum_{i \leq n} \Gamma_i(p)$. On the almost-sure event of Lemma 7 for $P_{1/j}$ the bound holds simultaneously for all $p \in P_{1/j}$, so the *function* $\bar{\Gamma}_n$ may be evaluated at the random $\hat{p}_n$:

$$\frac{1}{n} \sum_{i \leq n} W_{i,n} h(\theta_i) = \bar{\Gamma}_n(\hat{p}_n) + o(1) \stackrel{\text{Lem. 8}}{=} \int_{\Theta} \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} d\theta + o(1) \quad \text{a.s.}, \quad (\text{A3})$$

using $\frac{1}{n} \sum_{i \leq n} \bar{q}_i = \bar{q}_n^*$. The middle term is the function $\bar{\Gamma}_n(\cdot)$ evaluated at $\hat{p}_n$ and must not be read as $\mathbb{E}_{i-1}[\psi_{\hat{p}_n}(\xi_i)]$: $\hat{p}_n$ depends on draws after i , is not $\mathcal{F}_{i-1}$ -measurable, and that conditional expectation is not what Lemma 7 controls. Split the integral against $\int \pi h L_{\epsilon_\infty}$:

$$\left| \int \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} - \int \pi h L_{\epsilon_\infty} r \right| \leq \|h\|_\infty \pi_+ |\Theta| \left[R L_K(1/j) |\epsilon_n - \epsilon_\infty| + \left\| \frac{\bar{q}_n^*}{\bar{q}_n} - r \right\|_\infty \right],$$

on the event $\{1/j \leq \epsilon_\infty\}$ of the localization above, using $|L_{\epsilon_n} - L_{\epsilon_\infty}| \leq L_K(1/j) |\epsilon_n - \epsilon_\infty|$ (Assumption 1 and Jensen; every ϵ_n and ϵ_∞ lies in $[1/j, \epsilon_0]$ there) and $L_{\epsilon_\infty} \leq 1$, and where $R < \infty$ bounds $\bar{q}_n^*/\bar{q}_n$ for all large n — not from any structural bound on $\tilde{q}_i$, of which we assume none, but because (9) sends that ratio uniformly to $r \leq r_+$. Both bracketed terms vanish, the first because $\epsilon_n \rightarrow \epsilon_\infty$ and the second by (9). Hence $\frac{1}{n} \sum_i W_{i,n} h(\theta_i) \rightarrow \int \pi h L_{\epsilon_\infty} r$ a.s. Applying this with $h = f$ and with $h \equiv 1$ and dividing — legitimate because $Z_r = \int \pi L_{\epsilon_\infty} r \geq r_- Z_{\epsilon_\infty} > 0$ — gives $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$ a.s. Corollary 2 is the case $r \equiv 1$. $\square$

Remark 1. Consistency used Assumption 5(i) only through (9) and $\epsilon_n \rightarrow \epsilon_\infty$, and nothing at all about the form of $\tilde{q}_i$. This is what makes the theorem a statement about the algorithm as implemented rather than about an idealized variant of it: any sampler whose draw mixture the reported denominator tracks, in the ratio sense of (9), is covered — the archive rule, the capped rejection and the underflow redraws included. It is also the content that earlier drafts’ condition on convergence to the *idealized* cumulative mixture was missing: that object is not what the argument needs, and on its own does not deliver it.

A.3 Proof of Theorem 3 and Corollary 4

Let $g := f - \pi_{\epsilon_\infty}^r(f)$, a fixed bounded function — Assumption 5(ii) makes ϵ_∞ and r , hence $\pi_{\epsilon_\infty}^r(f)$, deterministic — satisfying $\int \pi g L_{\epsilon_\infty} r = 0$. Since $\sum_i W_{i,n} g(\theta_i) = (\sum_i W_{i,n})(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f))$ identically and $\frac{1}{n} \sum_i W_{i,n} \rightarrow Z_r > 0$ a.s. (Theorem 1 with $h \equiv 1$), by Slutsky’s theorem — which preserves stable convergence — it suffices to prove

$$S_n := \frac{1}{\sqrt{n}} \sum_{i \leq n} \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)} \longrightarrow \mathcal{N}(0, v^2) \text{ stably,} \quad v^2 := \int_{\Theta} \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\bar{q}_\infty^2} d\theta, \quad (\text{A4})$$

after which $\sigma_f^2 = v^2 / Z_r^2$, which is (13).

Step 1 (replacing the parameters by predictable proxies). $\bar{q}_\infty$ and ϵ_∞ are in general only $\mathcal{F}_\infty$ -measurable, so an array built from them is *not* adapted and Lemma 8 does not apply to it. We therefore centre on the predictable pair instead: let $\bar{q}_{i-1} \in \mathcal{Q}$ be the denominator (7) formed from the first $i-1$ draws and ϵ_{i-1} the bandwidth in force at the i -th proposal, both $\mathcal{F}_{i-1}$ -measurable, and put

$$V_i := \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_{i-1}}(\rho_i)}{\bar{q}_{i-1}(\theta_i)}.$$

Since $\bar{q}_n, \bar{q}_{i-1} \geq \delta\pi_-$ and $K \leq 1$, and $|\epsilon_n - \epsilon_{i-1}| \leq |\epsilon_n - \epsilon_\infty| + |\epsilon_{i-1} - \epsilon_\infty|$ (likewise for $\bar{q}$),

$$\left| S_n - \frac{1}{\sqrt{n}} \sum_{i \leq n} V_i \right| \leq C \sqrt{n} (|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) + \frac{C}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty)$$

with $C = \|g\|_\infty \pi_+ \max\{L_K(\epsilon_\infty)/(\delta\pi_-), (\delta\pi_-)^{-2}\}$ — a single constant here, since Assumption 5(ii) makes ϵ_∞ deterministic, so the central limit theorem needs no localization, and both brackets vanish by (10). This step is what the rates buy; the remainder of the proof never touches $\bar{q}_n$ or ϵ_n again.

Step 2 (drift). Lemma 8 with $\ell = 1$, $\phi = \pi g$ applies to V_i because $(\epsilon_{i-1}, \bar{q}_{i-1})$ is $\mathcal{F}_{i-1}$ -measurable and g is a fixed function under the central-limit clause of

Assumption 5; with $\int \pi g L_{\epsilon_\infty} r = 0$,

$$c_i := \mathbb{E}_{i-1} V_i = \int \pi g \left(L_{\epsilon_{i-1}} \frac{\tilde{q}_i}{\tilde{q}_{i-1}} - L_{\epsilon_\infty} r \right) d\theta,$$

$$|c_i| \leq C' (|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\tilde{q}_{i-1} - \tilde{q}_\infty\|_\infty),$$

C' depending only on $\|g\|_\infty, \pi_\pm, \delta, |\Theta|, L_K(\epsilon_\infty)$. The second half of (10) is exactly the statement that $n^{-1/2} \sum_{i \leq n}$ of that bound tends to zero, so $n^{-1/2} \sum_{i \leq n} c_i \rightarrow 0$. (Under the sufficient condition that each of the three terms is $O(i^{-1/2-a})$ this holds in each of the cases $a < 1/2$, $a = 1/2$, $a > 1/2$, where the partial sums are $O(n^{1/2-a})$, $O(\log n)$ and $O(1)$ respectively.)

Step 3 (martingale CLT). Let $X_{n,i} := n^{-1/2}(V_i - c_i)$, a martingale difference array with respect to the filtration $(\mathcal{F}_i)$, which does not depend on n and is therefore nested. Two conditions: (a) *Conditional Lindeberg.* $|X_{n,i}| \leq 2\|g\|_\infty \pi_+ / (\delta \pi_- \sqrt{n})$ surely, so for every $\eta > 0$ the truncated sum $\sum_i \mathbb{E}_{i-1} [X_{n,i}^2 \mathbf{1}\{|X_{n,i}| > \eta\}]$ is exactly zero for all n large. The defensive floor makes this step trivial where an unbounded-weight scheme would need a uniform-integrability argument. (b) *Conditional variance.* Lemma 8 with $\ell = 2$, $\phi = \pi^2 g^2$ gives

$$\mathbb{E}_{i-1} V_i^2 = \int \frac{\pi^2 g^2 L_{\epsilon_{i-1}}^{(2)}}{\tilde{q}_{i-1}^2} \tilde{q}_i d\theta = v^2 + R_i,$$

$$v^2 = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\tilde{q}_\infty^2} d\theta, \quad |R_i| \leq C'' (|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\tilde{q}_{i-1} - \tilde{q}_\infty\|_\infty),$$

where C'' carries $2L_K(\epsilon_\infty)$ for the squared kernel, since $|K_\epsilon^2 - K_{\epsilon'}^2| = |K_\epsilon - K_{\epsilon'}| (K_\epsilon + K_{\epsilon'}) \leq 2L_K(\epsilon_\infty) |\epsilon - \epsilon'|$. The bound on $|R_i|$ has vanishing Cesàro average, a fortiori from (10), so $\frac{1}{n} \sum_{i \leq n} \mathbb{E}_{i-1} V_i^2 \rightarrow v^2$ a.s.; with $\frac{1}{n} \sum_i c_i^2 \rightarrow 0$ from Step 2, $\sum_i \mathbb{E}_{i-1} X_{n,i}^2 = \frac{1}{n} \sum_i (\mathbb{E}_{i-1} V_i^2 - c_i^2) \rightarrow v^2$ a.s. Note that v^2 is permitted to be random here — an $\mathcal{F}_\infty$ -measurable limit of the conditional variances is exactly the setting the martingale CLT below covers, and it is only inside $\mathbb{E}_{i-1}$ that predictability was needed. The martingale central limit theorem for arrays with nested σ -fields, in its conditional-variance form [25, Cor. 3.1, pp. 58–59], then gives $\sum_i X_{n,i} \rightarrow \mathcal{N}(0, v^2)$ stably; the convergence is stable rather than merely distributional precisely because v^2 is $\mathcal{F}_\infty$ -measurable and possibly random. Combining Steps 1–3 gives (A4). $\square$

Proof of Corollary 4 Write $\hat{\sigma}_n^2 = [\frac{1}{n} \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2] / [\frac{1}{n} \sum_i W_{i,n}]^2$. In the numerator, $(f - \hat{\pi}_n(f))^2 = g^2 + 2g(\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f)) + (\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f))^2$, and $W_{i,n} \leq 1/\delta$ with $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$ a.s. (Theorem 1), so the cross terms vanish and it suffices to treat $\frac{1}{n} \sum_i W_{i,n}^2 g(\theta_i)^2$. Apply Lemma 7 on P_a with $a = \epsilon_\infty$ — deterministic here, so no localization is needed — to the class $\psi_{(\epsilon, p_q)}(\theta, \rho) = \pi(\theta)^2 g(\theta)^2 K_\epsilon(\rho)^2 / q_{p_q}(\theta)^2$, which is bounded by $\|g\|_\infty^2 / \delta^2$ and Lipschitz in (ϵ, p_q) by the argument of (A2) with $2L_K(\epsilon_\infty)$ in place of L_K ; the substitution of $\hat{p}_n$ used in (A3), followed by Lemma 8 with $\ell = 2$, $\phi = \pi^2 g^2$ (a fixed function under Assumption 5(ii)), gives $\frac{1}{n} \sum_i W_{i,n}^2 g^2(\theta_i) = \int \pi^2 g^2 L_{\epsilon_n}^{(2)} (\tilde{q}_n^* / \tilde{q}_n) \tilde{q}_n^{-1} + o(1)$, and $\epsilon_n \rightarrow \epsilon_\infty$, $\|\tilde{q}_n^* / \tilde{q}_n - r\|_\infty \rightarrow 0$ and $\|\tilde{q}_n - \tilde{q}_\infty\|_\infty \rightarrow 0$ (the last from (10)) send this to $\int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} r / \tilde{q}_\infty = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \tilde{q}_\infty / \tilde{q}_\infty^2 = v^2$ a.s., using $r = \tilde{q}_\infty / \tilde{q}_\infty$. The denominator tends

to Z_r^2 , so $\hat{\sigma}_n^2 \rightarrow v^2/Z_r^2 = \sigma_f^2$ a.s. Because the convergence in Theorem 3 is stable, the pair $(\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)), \hat{\sigma}_n)$ converges jointly to $(\sigma_f \mathcal{Z}, \sigma_f)$ — stable convergence is exactly joint convergence with every $\mathcal{F}_\infty$ -measurable variable, and σ_f is one — so with $\sigma_f > 0$ a.s. the continuous mapping theorem gives $\mathcal{Z} \sim \mathcal{N}(0, 1)$ as the limit of the ratio. Marginal convergence to a mixed normal alone would not suffice. $\square$

Remark 2 (growing snapshot count). Nothing above requires m to be fixed except the compactness and fixed dimension D of P_q — and the fixed floor δ , which the implementation ties to m as $0.5/(m+1)$ and which must be decoupled before m is allowed to grow, since Lemma 6’s bound is $1/\delta$. With δ fixed and $m = m_n \rightarrow \infty$, run the proof of Lemma 7 with an n -dependent net: the union bound costs $D_n \log(1/\eta_n)$ with $D_n = \Theta(m_n)$, and the Lipschitz constant over the mixture weights grows at most polynomially in m_n , so choosing $\eta_n \rightarrow 0$ fast enough leaves the Borel–Cantelli step summable for any $m_n = o(n/\log n)$. In that regime the snapshot clause of Assumption 5 follows from $V_n/m_n \rightarrow 0$, with $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ the total variation of the proposal path, rather than having to be assumed: the draw-proportional m_n -point mixture is then a converging quadrature of the empirical mixture. The retroactive pass costs $\mathcal{O}(n m_n k)$.

A.4 Proof of Proposition 5

Write $\zeta_0 := \pi_{\epsilon_\infty}(|r - 1|)$. Under the hypothesis, $\zeta_0 \leq \eta_A + \pi_{\epsilon_\infty}(A^c) = \zeta$. For bounded f , since $\pi_{\epsilon_\infty}^r(f) = \pi_{\epsilon_\infty}(fr)/\pi_{\epsilon_\infty}(r)$,

$$\pi_{\epsilon_\infty}^r(f) - \pi_{\epsilon_\infty}(f) = \frac{\pi_{\epsilon_\infty}(f(r-1)) - \pi_{\epsilon_\infty}(f)\pi_{\epsilon_\infty}(r-1)}{\pi_{\epsilon_\infty}(r)},$$

whose modulus is at most $2\|f\|_\infty \zeta_0/(1 - \zeta_0)$ because $\pi_{\epsilon_\infty}(r) \geq 1 - \zeta_0$. Taking the supremum over $\|f\|_\infty \leq 1$ and halving bounds the total variation by $\zeta_0/(1 - \zeta_0)$; as $x \mapsto x/(1 - x)$ increases on $[0, 1)$, this is at most $\zeta/(1 - \zeta)$ whenever $\zeta < 1$, which is (14). For $\zeta \geq 1$ the bound degenerates and only the trivial $\|\cdot\|_{\text{TV}} \leq 1$ survives.

The closed form quoted in §4.1(a) for the prior floor’s contribution follows by evaluating (9) for that contribution alone: with $u = q_\infty/\pi$ and a limiting bootstrap share $\nu_\infty < \delta$,

$$r_{\text{floor}} - 1 = \frac{\nu_\infty + (1 - \nu_\infty)u}{\delta + (1 - \delta)u} - 1 = \frac{(\delta - \nu_\infty)(u - 1)}{\delta + (1 - \delta)u},$$

so on $\{u \geq c\}$, splitting at $u = 1$ and bounding each branch, $|r_{\text{floor}} - 1| \leq (\delta - \nu_\infty) \max\{(1 - \delta)^{-1}, (\delta + (1 - \delta)c)^{-1}\}$, which is the η_A of Proposition 5 for $A = \{u \geq c\}$. It is identically 1 when $\nu_\infty \geq \delta$, and $\rightarrow 1$ as $\delta \rightarrow 0$. $\square$

A.5 Status of each assumption in the implementation

Appendix B Implementation Details

Tolerance schedulers.

Every scheduler is a pure function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum ϵ_{hist} (§3.2), so the effective tolerance can only decrease. Let μ denote the extra-individual margin (default $\mu = k$); it is unrelated to the snapshot count m of §4. The three base rules act on the individuals *within* the current bandwidth, i.e. the hard-threshold set $\{i : \rho_i < \epsilon\}$. (This bandwidth-membership count is used *only* to drive the tolerance schedule; it is distinct from the smooth-kernel weight $K_\epsilon(\rho_i)$ that defines the proposal mixture and the reported posterior, for which every evaluated particle contributes continuously. We avoid the word “accepted” here precisely because acceptance is probabilistic under a smooth kernel.):

- **Quantile.** $\epsilon \leftarrow$ the lower-rank (non-interpolated) p -th percentile of the accepted losses, default $p = 50$; applied only once at least $k + m$ individuals are accepted.
- **Geometric decay.** $\epsilon \leftarrow \epsilon_0 \gamma^e$ after each completed epoch e of $k + \mu$ accepted individuals, default $\gamma = 0.9$.
- **Acceptance rate.** Over a sliding window of the most recent $k + \mu$ individuals, if the acceptance rate exceeds r_{hi} (default 0.3) tighten by a factor 0.9, otherwise hold; the rule never expands ϵ (the low-rate guard $r_{\text{lo}} = 0.1$ simply holds).

With a smooth (Gaussian or Epanechnikov) kernel the schedulers run in a *kernel-aware* mode: instead of the hard $\rho < \epsilon$ count, ϵ is chosen by bisection so that the per-step kernel-weighted effective-sample-size retention ratio meets a target (default 0.95), which keeps the bandwidth schedule comparable across kernel choices.

Proposal covariance.

The archive A_n is the top- k history members by smallest discrepancy. Component weights are formed in log-space as $\tilde{W}_j \propto w_j K_{\epsilon_n}(\rho_j)$ and normalised (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is $\Sigma_n = s[\widehat{\text{Cov}}_{\tilde{W}}(A_n) + \lambda_n I]$ with s the `perturbation_scale` and a two-part jitter $\lambda_n = 10^{-9} \text{tr} \widehat{\text{Cov}}_{\tilde{W}}(A_n)/d + 10^{-12} \bar{b}^2$, where $\bar{b}^2$ is the mean squared prior-box width; it is Cholesky-factored once per call. The first term is scale-aware, so a tightly converged archive (marginal standard deviation $\sim 10^{-4}$) is not swamped by the floor as a fixed 10^{-6} would swamp it; the second is a run-independent absolute floor that keeps the matrix factorable for a degenerate archive and supplies the uniform lower spectral bound Assumption 2 needs (Table A1). Each mixture component is truncated to the prior box, with its log normalising mass computed as the product of the per-axis in-box masses. That is exact for a diagonal covariance, in which case the proposal integrates to one on the support; for a correlated covariance it is an approximation, so the evaluated proposal is a positive function that need not integrate to one — which is why §4 states its denominator assumption for positive functions and books the discrepancy as factor (c) of r (§4.1).

Matched pyABC acceptor.

For the synchronous baseline, pyABC’s `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy ρ with probability $K_\epsilon(\rho)$ (peak-normalised so $K_\epsilon(0) = 1$), using the *same* kernel K_ϵ as the asynchronous side; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5).

Bounded-memory posterior estimator.

The retroactive AMIS estimator (8) evaluates the snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \leq S+1 = 21$ history-reconstructed proposals with a defensive prior component (mass floored at $0.5/(m+1)$) — at each of the n evaluated particles, at total cost $\mathcal{O}(nmk) = \mathcal{O}(nk)$ with m fixed. The snapshots are taken draw-proportionally from the reconstructed archive (indices $0, \text{step}, 2\text{step}, \dots$ with $\text{step} = \max(1, \lfloor |\text{archive}|/S \rfloor)$), plus the most recent archive-phase proposal if it is not already among them — hence $m \leq S+1 = 21$ rather than S . For near-instantaneous simulators the history reaches $\mathcal{O}(10^7)$ particles, so the pass is accumulated in chunks of $2^{16} = 65,536$ history points: peak memory is $\mathcal{O}(\text{chunk} \cdot m \cdot k)$ rather than $\mathcal{O}(n)$. Each particle’s denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across* chunk boundaries), so chunking changes memory, not the value, and the result is bit-identical to the unchunked computation. Measured on the largest history (Gaussian-mean, $n \approx 3 \times 10^7$ evaluated particles, $k = 100$, $m = 20$), the one-time retroactive pass runs in ≈ 20 minutes on a single core with ≈ 0.3 GB peak memory (independent of n ; the intermediate is one 2^{16} -point chunk) and parallelizes trivially across evaluation points; it runs after the timed budget and is excluded from the throughput measurements by design. For a near-instantaneous simulator this is comparable to the timed budget itself — non-trivial though bounded — whereas for cost-bearing simulators, whose histories are orders of magnitude shorter, it is negligible against the simulation time.

Appendix C Additional Experimental Protocol

Effective simulating ranks.

A “ W -worker” point allocates W physical cores to both arms, but the two arms use them differently. Every rank of the asynchronous method evaluates, so it has W simulating ranks. pyABC’s MPI sampler is root-driven: rank 0 distributes work to ranks $1, \dots, W-1$ and does not simulate, so the baseline has $W-1$. We report the allocated core count W for both, which flatters the asynchronous arm by $1/W$ — 2.1% at $W=48$, 0.3% at $W=384$. We do not correct for it because the correction shrinks monotonically with scale while the gap we report grows with scale, so it cannot account for any reported effect; but it should be read as part of the coordination cost the barrier-free design avoids rather than as a measurement we have netted out.

Seeding.

Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`--extend`) runs.

Crash recovery (kill-and-resume).

We test the crash-recoverability claim of §3.1 directly. On the Gaussian-mean benchmark (4 workers, a 1.2×10^5 -simulation budget) we compare a clean uninterrupted run against a run that is hard-killed (SIGKILL of the entire worker process group) partway through inference and then relaunched with an identical command; recovery reloads the periodic population checkpoint and resumes from the next generation. The resumed run recovers the analytic posterior mean — absolute error 7.6×10^{-3} , against 7.5×10^{-4} for the clean run — and completes the same total simulation budget with no duplicated records. It is *not* a bit-identical replay: the two reported posterior means differ by 6.9×10^{-3} . Three things prevent the replay, and they are worth separating. Two are incidental — the candidate generator is re-seeded from its (replicate, rank) key on restart rather than checkpointed, and MPI arrival order is not deterministic. The third is structural: the online AMIS snapshot buffer is rebuilt empty on restart, and since it enters the importance denominator of every subsequent candidate, and those weights form the archive’s proposal mixture, the post-restart proposal sequence is drawn from a different (though equally valid) adaptation trajectory (§3.1). Checkpointing the buffer would remove the third but not the first two. This is exactly the sense in which the method is “reproducible up to MPI arrival order” (§3.1): recovery yields a *valid* posterior trajectory over the actual evaluated history, not the identical one. In this instance the kill landed *during* a checkpoint write and truncated the primary checkpoint; recovery fell back to the previous-state backup that the dumper writes before overwriting, so the resume succeeded automatically — the reported estimator, being a pure function of the recovered history, is unaffected by which of the two checkpoints is used.

Cluster and wall-clock control.

Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All wall-clock-budgeted methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serialises its current state into the records and returns, so a partial final generation is recorded rather than discarded (the simulation-limited studies marked † in Table C2 — the SBC and kill-and-resume runs — instead stop at a fixed per-trial simulation count). Posterior estimation and plotting run off the timed inference path, so they do not count against the budget.

Benchmark configurations.

Table C2 lists the resolved settings (k is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size $k \in \{100, 1000\}$; all other studies fix the workers shown.

Benchmark-specific notes.

Each benchmark fixes one observed dataset (generated from seed 42) and infers parameters under uniform priors; the discrepancy is a distance between simulated and observed summary statistics. *Gaussian mean*: $n_{\text{obs}} = 100$ observations $\sim \mathcal{N}(0, 1)$, prior $\mu \sim \mathcal{U}[-5, 5]$, summary the sample mean, discrepancy $|\bar{x}_{\text{sim}} - \bar{x}_{\text{obs}}|$; the closed-form Gaussian posterior is the calibration and recovery reference. *g-and-k*: the quantile-defined g-and-k distribution ($c = 0.8$), $n_{\text{obs}} = 1000$, true $(A, B, g, k) = (3, 1, 2, 0.5)$, priors $A \sim \mathcal{U}[0, 6]$, $B \sim \mathcal{U}[0.1, 4]$, $g \sim \mathcal{U}[0, 5]$, $k \sim \mathcal{U}[0, 1]$; summary the seven octiles (the 0.125, ..., 0.875 quantiles), discrepancy Euclidean. *Lotka–Volterra*: a Gillespie stochastic simulation of the four-reaction predator–prey system from $(X_0, Y_0) = (50, 100)$ over $T = 30$, true rates $\theta = (0.5, 0.025, 0.025, 0.5)$, priors $\theta_1, \theta_4 \sim \mathcal{U}[0.05, 2]$ and $\theta_2, \theta_3 \sim \mathcal{U}[0.001, 0.2]$; summary the six statistics $(\bar{X}, \text{sd } X, \bar{Y}, \text{sd } Y, \log(1 + \text{var } X/\bar{X}), \log(1 + \text{var } Y/\bar{Y}))$, discrepancy Euclidean. *Cellular Potts*: the real `cellsInSilico` (NASTJA) simulator, inferring the cell division rate and motility on normalized $[0, 1]$ priors that map to physical ranges $[6 \times 10^{-5}, 0.6]$ and $[0, 10^4]$ (true normalized values 0.0499 and 0.2); the discrepancy is a whitened distance over morphological summary features of the simulated cell configuration (radial density and pair-correlation profiles, free-area and structure-factor profiles, a DBSCAN dispersion fraction, and log cell count and radius) against reference summaries. Its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

Computing environment.

All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and ≈ 94 GB memory per node, connected by Mellanox EDR InfiniBand. The memory-heavy fast-simulator reruns instead used the 180 GB high-memory partition. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT for the sliced Wasserstein distance; the asynchronous method runs as a propagator in our fork of Propulate. Unless noted, ranks are placed one per physical core (`srun --ntasks-per-node=48 --cpus-per-task=1`), so a “ W -worker” point occupies W physical cores packed onto $\lceil W/48 \rceil$ fully-reserved nodes; the memory-heavy Gaussian-family reruns instead placed 24 ranks per high-memory node under whole-node reservation. Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

References

- [1] Beaumont, M.A.: Approximate bayesian computation. *Annual Review of Statistics and Its Application* **6**, 379–403 (2019) <https://doi.org/10.1146/annurev-statistics-030718-105212>
- [2] Sisson, S.A., Fan, Y., Tanaka, M.M.: Sequential monte carlo without likelihoods. *Proceedings of the National Academy of Sciences* **104**(6), 1760–1765 (2007) <https://doi.org/10.1073/pnas.0607208104>
- [3] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface* **6**(31), 187–202 (2009) <https://doi.org/10.1098/rsif.2008.0172>
- [4] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate bayesian computation. *Biometrika* **96**(4), 983–990 (2009) <https://doi.org/10.1093/biomet/asp052>
- [5] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential monte carlo method for approximate bayesian computation. *Statistics and Computing* **22**(5), 1009–1020 (2012) <https://doi.org/10.1007/s11222-011-9271-y>
- [6] Taubert, O., Weißer, M., Kelle, D., Bayer, M., Knüttel, H., Comito, C., Streit, A., Götz, M.: Massively parallel genetic optimization through asynchronous propagation of populations. In: *International Conference on High Performance Computing (ISC High Performance)*, pp. 106–124 (2023). https://doi.org/10.1007/978-3-031-32041-5_6
- [7] Wilkinson, R.D.: Approximate bayesian computation (abc) gives exact results under the assumption of model error. *Statistical Applications in Genetics and Molecular Biology* **12**(2), 129–141 (2013) <https://doi.org/10.1515/sagmb-2013-0010>
- [8] Fearnhead, P., Prangle, D.: Constructing summary statistics for approximate bayesian computation: semi-automatic approximate bayesian computation. *Journal of the Royal Statistical Society: Series B* **74**(3), 419–474 (2012) <https://doi.org/10.1111/j.1467-9868.2011.01010.x>
- [9] Cornuet, J.-M., Marin, J.-M., Mira, A., Robert, C.P.: Adaptive multiple importance sampling. *Scandinavian Journal of Statistics* **39**(4), 798–812 (2012) <https://doi.org/10.1111/j.1467-9469.2011.00756.x>
- [10] Klinger, E., Rickert, D., Hasenauer, J.: pyabc: distributed, likelihood-free inference. *Bioinformatics* **34**(20), 3591–3593 (2018) <https://doi.org/10.1093/bioinformatics/bty361>

- [11] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyabc 0.12.3: high-performance distributed, likelihood-free inference. *Bioinformatics* **38**(13), 3354–3355 (2022) <https://doi.org/10.1093/bioinformatics/btac321>
- [12] Drovandi, C.C., Pettitt, A.N.: Estimation of parameters for macroparasite population evolution using approximate bayesian computation. *Biometrics* **67**(1), 225–233 (2011) <https://doi.org/10.1111/j.1541-0420.2010.01410.x>
- [13] Lenormand, M., Jabot, F., Deffuant, G.: Adaptive approximate bayesian computation for complex models. *Computational Statistics* **28**(6), 2777–2796 (2013) <https://doi.org/10.1007/s00180-013-0428-3>
- [14] Veach, E., Guibas, L.J.: Optimally combining sampling techniques for monte carlo rendering. In: *Proceedings of SIGGRAPH '95*, pp. 419–428 (1995). <https://doi.org/10.1145/218380.218498>
- [15] Cappé, O., Guillin, A., Marin, J.-M., Robert, C.P.: Population Monte Carlo. *Journal of Computational and Graphical Statistics* **13**(4), 907–929 (2004) <https://doi.org/10.1198/106186004X12803>
- [16] Marin, J.-M., Pudlo, P., Sedki, M.: Consistency of adaptive importance sampling and recycling schemes. *Bernoulli* **25**(3), 1977–1998 (2019) <https://doi.org/10.3150/18-BEJ1042>
- [17] Bugallo, M.F., Elvira, V., Martino, L., Luengo, D., Míguez, J., Djurić, P.M.: Adaptive importance sampling: The past, the present, and the future. *IEEE Signal Processing Magazine* **34**(4), 60–79 (2017) <https://doi.org/10.1109/MSP.2017.2699226>
- [18] Vergé, C., Dubarry, C., Del Moral, P., Moulines, E.: On parallel implementation of sequential Monte Carlo methods: The island particle model. *Statistics and Computing* **25**(2), 243–260 (2015) <https://doi.org/10.1007/s11222-013-9429-x>
- [19] Paige, B., Wood, F., Doucet, A., Teh, Y.W.: Asynchronous anytime sequential monte carlo. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27 (2014)
- [20] Murray, L.M., Lee, A., Jacob, P.E.: Parallel resampling in the particle filter. *Journal of Computational and Graphical Statistics* **25**(3), 789–805 (2016) <https://doi.org/10.1080/10618600.2015.1062015>
- [21] Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., Mira, A.: Abcpy: A high-performance computing perspective to approximate bayesian computation. *Journal of Statistical Software* **100**(7), 1–38 (2021) <https://doi.org/10.18637/jss.v100.i07>
- [22] Lintusaari, J., Vuollekoski, H., Kangasräsiö, A., Skytén, K., Järvenpää, M.,

- Marttinen, P., Gutmann, M.U., Vehtari, A., Corander, J., Kaski, S.: ELFI: Engine for likelihood-free inference. *Journal of Machine Learning Research* **19**(16), 1–7 (2018)
- [23] Alamoudi, E., Reck, F., Bundgaard, N., Graw, F., Brusch, L., Hasenauer, J., Schälte, Y.: A wall-time minimizing parallelization strategy for approximate Bayesian computation. *PLOS ONE* **19**(2), 0294015 (2024) <https://doi.org/10.1371/journal.pone.0294015>
- [24] Owen, A., Zhou, Y.: Safe and effective importance sampling. *Journal of the American Statistical Association* **95**(449), 135–143 (2000) <https://doi.org/10.1080/01621459.2000.10473909>
- [25] Hall, P., Heyde, C.C.: *Martingale Limit Theory and Its Application*. Academic Press, New York (1980)

Table A1 What the implementation guarantees, what it assumes, and what it violates. “By construction” means the code enforces the property for every run, not that it was checked post hoc.

Assumption	Status	Basis
1 kernel, Lipschitz part	by construction	Gaussian and Epanechnikov are peak-normalized and Lipschitz in ϵ on every $[a, \epsilon_0]$, with $L_K(a) = 2/ea$ and $2/a$. The hard indicator is not, and is excluded from the theory.
1 $Z_{\epsilon_\infty} > 0$	holds for Gaussian	The Gaussian kernel is strictly positive, so $L_{\epsilon_\infty} > 0$ and the limiting target is non-degenerate whatever the discrepancy distribution. For a compactly supported kernel (Epanechnikov) it requires the achievable discrepancies to reach inside the support, which we do not verify.
2 denominator class	by construction	The prior support is a box and the archive lies in it, so component means do; the covariance carries an additive jitter $10^{-9}\text{tr}(\hat{\Sigma})/d + 10^{-12}\bar{b}^2$ ($\bar{b}^2$ the mean squared box width), whose second term is a run-independent floor, giving $\sigma_-^2 > 0$, and σ_+^2 follows from the archive lying in a bounded box. The truncation <i>normalizer</i> is the product of per-axis in-box masses, so with a correlated covariance $\bar{q}_n$ does not integrate to one — which Assumption 2 does not require: it constrains a positive function, and the discrepancy is carried by r instead (§4.1(c)).
4 adapted filtration	assumed	Automatic for a sequential run. Under asynchrony a discrepancy may be in flight when the next candidate is proposed, so the field of realized outcomes cannot serve; the assumption is that a larger filtration exists under which the candidate’s conditional law is still $\bar{q}_i$. Where completion times steer scheduling, $\bar{q}_i$ is that enlarged-field law rather than the emitted proposal density. Same mechanism as Limitation (iv).
3 prior floor	by construction	The retroactive denominator floors the prior component’s mass at $\delta = 0.5/(m+1)$ (Appendix B), so $\bar{q}_n \geq \delta\pi$ and Lemma 6 bounds every reported weight by $1/\delta$ — independently of what the sampler does.
5(i) fidelity (9)	assumed; two of its four factors measured	The one place the sampler enters for Theorem 1. Its four factors are decomposed in §4.1; two of them — the prior floor and the snapshot quadrature — are estimated from stored histories rather than bounded ($\hat{\zeta} = 0.022$ on the Cellular Potts production run, the reported posterior 1.1% in total variation from a large- m reference), while the truncation constant and the rejection/redraw conditioning are bounded only. Since the factors multiply, the measured part is a partial accounting rather than a bound on the whole.
5(ii) stabilization	violated as configured	Not implied by the top- k rule, and now measured rather than left open: on the Cellular Potts run the per-step drift decays as $\tau^{-0.92}$ (archive turnover 832 changes in 33,748 draws, the $O(k \log n)$ record rate), while the distance to the limit decays only as τ^{-b} , $b \approx 0.14$ – 0.33 , against the $b > \frac{1}{2}$ needed. A serial benchmark passes at $b \approx 0.6$ – 0.76 when the target is identified and fails at $b \approx 0.0$ – 0.29 when its parameters confound, which is the Cellular Potts case by construction — so the condition is one on the inference problem, not on the sampler. The central limit theorem therefore covers the damped or frozen variants, not this configuration; consistency, needing no rate, is unaffected. The asynchronous replay of the proposal sequence enters here too, since a reporting rank’s arrival-ordered log need not reproduce the proposal each candidate came from.
5 $\epsilon_\infty > 0$ (deterministic, for the CLT only)	not satisfied	A strictly positive <code>min_tol</code> forces $\epsilon_\infty \geq \text{min_tol} > 0$, since $\{\epsilon_n\}$ is non-increasing and clipped from below, which is all Theorem 1 needs. Theorem 3 additionally needs ϵ_∞ deterministic, so that $f - \pi_{\epsilon_\infty}(f)$ is a fixed function; a floor alone does not give that, since a monotone data-driven schedule may settle above it at a run-dependent value. A schedule that reaches its floor in finite time does, and makes the ϵ terms of (10) vanish identically. The runs reported here leave <code>min_tol</code> unset (zero is accepted), so neither part is <i>established</i> — not that either is disproved: a data-driven decreasing schedule may well converge to a positive limit, and may even converge to a deterministic one; we simply do not verify it. Note also that the bandwidth a finite budget reaches is not itself an asymptotic ϵ_∞ .
5 rate (10)	assumed	Needed only in Step 1 and Step 2 of the proof of Theorem 3; consistency does not use it.

Table C2 Resolved experiment configurations. Workers, per-run wall-clock budget, replicates (trials for SBC), archive size k , and methods (A: asynchronous; S: matched-kernel synchronous baseline; R: rejection ABC). [†] Simulation-limited: the budget is per-trial (SBC) or per-run (kill-and-resume) *simulations*, not wall-clock seconds. [‡] Also run with AMIS on/off, a drain-after-deadline variant, and a steeper coupling regime (base delay 0.5 s, cap 8 s).

Experiment	Workers	Budget (s)	Reps	k	Methods
Gaussian mean	48	300	5	100	A,S,R
g-and-k	48	600	5	100	A,S,R
Lotka–Volterra	48	600	5	100	A,S
Cellular Potts	48	3600	5	100	A,S,R
Straggler	16	300	5	100	A,S
Runtime heterogeneity	48	60	5	100	A,S
Parameter-coupled runtime [‡]	48	60	5	100	A,S
Strong scaling (LV)	1–288	180	5	100/1000	A,S
Strong scaling (CPM)	1–384	1800	3–5	100	A,S
Sensitivity	48	300	5	50/200	A
Ablation	48	300	5	100	A
SBC (Gaussian, 1-D)	48	20k [†]	1000	100	A,S
SBC (bimodal)	48	20k [†]	1000	100	A,S
SBC (g-and-k, 4-D)	48	40k [†]	1000	200	A,S
SBC (g-and-k, 2×)	48	80k [†]	500	200	A,S
Kill-and-resume	4	120k [†]	1	100	A